

Java 25 — Module Import Declarations

Aspect	Details
Feature	Module Import Declarations
UniqueClassName	ModuleImportDeclarations
JEP	JEP 494
Java Version	25
Year	2025
Status	Final
Main syntax	import module java.base;
Purpose	Import all accessible top-level classes and interfaces exported by a module
Primary benefit	Reduces large numbers of individual import declarations, especially for beginner-friendly and compact Java programs

Module import declarations became a **final feature in Java 25** after previewing in Java 23 and Java 24. The feature is particularly useful when a program wants to use many APIs from a module without listing every individual type.

1. What Are Module Import Declarations?

Traditionally, Java imports individual types:

```
import java.util.List;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;
```

Java 25 allows:

```
import module java.base;
```

This makes the **accessible exported top-level types** from the module available for simple-name use.

For example:

```
import module java.base;

public class Main {
    public static void main(String[] args) {
        List<String> names = new ArrayList<>();

        names.add("Anurag");
        names.add("Java");

        System.out.println(names);
    }
}
```

Instead of:

```
import java.util.List;
import java.util.ArrayList;
```

you can use the module import.

Concept

Traditional imports

```
import java.util.List;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.Map;
```



Individual types

Module import

```
import module java.base;
```



Accessible exported types
from java.base

2. Advantages

Advantage	Explanation	Real-world benefit
Fewer import statements	One module import can replace many individual imports	Less boilerplate

Cleaner small programs	Particularly useful for simple applications and examples	Improves readability
Beginner-friendly	Beginners don't need to understand exactly which package contains every commonly used class	Easier Java learning
Useful with java.base	Many fundamental Java APIs are in java.base	Convenient for small programs
Good for educational code	Tutorials can focus on Java concepts rather than import management	Better teaching experience
Useful with compact source files	Fits well with Java's effort to make simple programs more concise	Less ceremony
Refactoring convenience	Adding another commonly used type from an imported module may not require another import line	Faster experimentation
Module-level abstraction	Expresses dependency at module level	Can make the intended module dependency clearer
Works with automatic module contents	The compiler determines which accessible types are available	Avoids manually maintaining long import lists
Complements other Java simplification features	Works naturally with modern concise Java source constructs	Useful for scripts/examples/prototypes

3. Disadvantages

Disadvantage	Explanation	Impact
Can import more names than needed	A module can expose many types	Less explicit code
Possible name conflicts	Two module imports can make identically named	May require explicit imports or qualified names

	types available	
Less precise dependency expression	import java.util.List communicates exactly what is used; module import is broader	Code reviewers have less immediate information
Potential readability issue in large applications	Large modules can expose many APIs	Individual imports may be clearer
Not a performance feature	It primarily affects source-code convenience	Doesn't make runtime execution faster
Doesn't import non-exported packages	Module encapsulation still applies	Internal APIs remain inaccessible
Doesn't automatically solve static imports	Static members have separate import rules	You may still need normal/static imports
Potential ambiguity	Multiple modules/packages may contain types with the same simple name	Explicit imports may be necessary
Not always appropriate for libraries	Library code benefits from very explicit dependencies	Broader imports can hide what the source actually uses
Requires Java 25+ for the final feature	Older Java versions don't support the final syntax	Limits source compatibility

4. Traditional Imports vs Module Import

Aspect	Traditional Import	Module Import Declaration
Syntax	import java.util.List;	import module java.base;
Import granularity	Individual type	Module
Number of imports	Potentially many	Usually fewer
Explicitness	★ ★ ★ ★ ★	★ ★ ★
Convenience	★ ★ ★	★ ★ ★ ★ ★
Small programs	Good	Excellent

Tutorials	Good	Excellent
Large enterprise applications	Excellent	Good
Library source code	Excellent	Usually preferable to stay explicit
Name collision control	Better	May require additional qualification
Runtime performance	Same	Same
Main purpose	Precise type selection	Reduce import boilerplate

5. When to Use Module Import Declarations

Situation	Use?	Reason
Small standalone Java program	✓ Yes	Reduces boilerplate
Educational examples	✓ Yes	Keeps focus on the Java concept
Java tutorials	✓ Yes	Students don't need many import statements
Code demonstrations	✓ Yes	Shorter examples
Prototypes	✓ Yes	Faster experimentation
Small command-line utilities	✓ Yes	Convenient when many standard APIs are used
Scratch/test programs	✓ Yes	Less ceremony
Using many APIs from one module	✓ Yes	Particularly useful
Simple java.base applications	✓ Yes	Many commonly used APIs come from java.base
Large application classes	□ Sometimes	Use when the broader import remains readable
Public libraries/frameworks	□ Usually no	Explicit imports communicate dependencies better

--	--	--

6. When Not to Use

Situation	Don't use?	Why	Better approach
Large enterprise class	☐ Usually	Too broad	Individual imports
API/library code	☐ Usually	Dependency intent should be explicit	Individual imports
Many modules with overlapping type names	✗	Higher ambiguity risk	Explicit imports
Code where imports document dependencies	✗	Module import hides individual dependencies	Individual imports
Strict coding standards require explicit imports	✗	Violates project conventions	Follow project style
Performance optimization	✗	No meaningful runtime performance improvement	Optimize actual runtime bottlenecks
Security isolation	✗	Import declarations don't provide security boundaries	Java modules/access control
Accessing internal JDK APIs	✗	Module imports don't bypass encapsulation	Use supported public APIs

7. Real-Time Use Case — Small Java Application

Traditional

```
import java.util.ArrayList;
import java.util.List;
import java.util.HashMap;
import java.util.Map;
```

```
public class CustomerApp {
```

```

public static void main(String[] args) {
    List<String> customers = new ArrayList<>();

    Map<Integer, String> customerMap = new HashMap<>();

    customers.add("Alice");
    customers.add("Bob");

    customerMap.put(1, "Alice");
    customerMap.put(2, "Bob");

    System.out.println(customers);
    System.out.println(customerMap);
}
}

```

With Module Import

```

import module java.base;

public class CustomerApp {

    public static void main(String[] args) {
        List<String> customers = new ArrayList<>();

        Map<Integer, String> customerMap = new HashMap<>();

        customers.add("Alice");
        customers.add("Bob");

        customerMap.put(1, "Alice");
        customerMap.put(2, "Bob");

        System.out.println(customers);
        System.out.println(customerMap);
    }
}

```

Benefit

```

3+ individual imports
    ↓
1 module import
    ↓
Cleaner source

```

8. Real-Time Use Case — Educational Program

Suppose you're teaching collections:

```
import module java.base;

public class CollectionDemo {

    public static void main(String[] args) {

        List<String> languages =
            new ArrayList<>();

        languages.add("Java");
        languages.add("Python");
        languages.add("Go");

        for (String language : languages) {
            System.out.println(language);
        }
    }
}
```

The student can concentrate on:

List
ArrayList
for-each

rather than first learning:

java.util.List
java.util.ArrayList

This makes module imports especially attractive for **teaching and introductory examples**.

9. Real-Time Use Case — Multiple APIs from java.base

A small utility may use:

java.lang
java.util
java.io
java.time
java.nio
java.net

Many of these are within java.base.

Instead of numerous imports:

```
import java.time.LocalDate;
import java.util.List;
import java.util.ArrayList;
import java.util.Optional;
```

you could use:

```
import module java.base;
```

Then:

```
LocalDate today = LocalDate.now();
```

```
List<String> values =
    new ArrayList<>();
```

```
Optional<String> result =
    Optional.of("Java");
```

This is one of the strongest practical cases for the feature.

10. Real-Time Use Case — Prototype

Imagine you're quickly testing an idea:

```
import module java.base;
```

```
public class Prototype {

    public static void main(String[] args) {

        var users = new ArrayList<String>();

        users.add("User-1");
        users.add("User-2");

        var timestamp = Instant.now();

        System.out.println(timestamp);
        System.out.println(users);
    }
}
```

You can experiment without spending time maintaining a large import section.

Good fit

Idea
↓
Prototype
↓
Experiment
↓
Validate

Later, production code can switch to more explicit imports if desired.

11. Real-Time Use Case — Command-Line Utility

A small command-line application may use:

```
import module java.base;

public class FileReporter {

    public static void main(String[] args) throws Exception {

        var path = Path.of("report.txt");

        var lines = Files.readAllLines(path);

        System.out.println("Lines: " + lines.size());
    }
}
```

This is convenient because several commonly used classes come from the same module.

12. Name Conflict Example

Module imports can become less attractive when different modules expose types with the same name.

For example, suppose two imported modules expose a type called:

Date

Then:

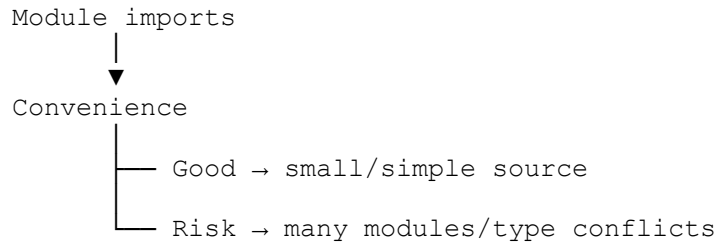
Date date;

may become ambiguous.

The solution is to explicitly import the desired type or use its fully qualified name.

```
import some.package.Date;
```

Therefore:



13. Module Import vs Wildcard Import

This is an important distinction.

Package wildcard

```
import java.util.*;
```

means:

Import accessible types from **one package**.

Module import

```
import module java.base;
```

means:

Make accessible exported top-level types from **the module's exported packages** available.

Feature	Package wildcard	Module import
Example	<pre>import java.util.*;</pre>	<pre>import module java.base;</pre>
Scope	Package	Module
Multiple packages	✗	✓

Boilerplate reduction	Moderate	High
Explicitness	Low	Lower
Potential conflicts	Moderate	Potentially higher
Best use	Package-focused code	Module-focused/simple code

14. Alternate Features / Approaches

There is no exact one-for-one replacement because Module Import Declarations solve a **source-code convenience** problem.

Requirement	Alternative	When preferable
Import a single class	<code>import java.util.List;</code>	Best for explicit production code
Import many classes from one package	<code>import java.util.*;</code>	When package-level convenience is acceptable
Import static members	<code>import static ...</code>	When static APIs need direct access
Avoid import completely	Fully qualified class name	When resolving ambiguity
Short simple examples	Module import	Best when many APIs from a module are used
Explicit dependency documentation	Individual imports	Best for libraries/enterprise code
Package-level convenience	Wildcard import	When all required types come from one package
Modern compact Java source	Module imports + other concise syntax	Excellent for educational/demo programs

15. Module Import vs Wildcard Import vs Explicit Import

Criterion	Explicit Import	Package Wildcard	Module Import
Example	<code>import java.util.List;</code>	<code>import java.util.*;</code>	<code>import module java.base;</code>

Precision	★ ★ ★ ★ ★	★ ★ ★	★ ★
Convenience	★ ★ ★	★ ★ ★ ★	★ ★ ★ ★ ★
Readability in large projects	★ ★ ★ ★ ★	★ ★ ★	★ ★ ★
Small programs	★ ★ ★ ★	★ ★ ★ ★	★ ★ ★ ★ ★
Educational examples	★ ★ ★	★ ★ ★ ★	★ ★ ★ ★ ★
Risk of name ambiguity	Low	Medium	Higher
Dependency visibility	Excellent	Moderate	Lower
Boilerplate	Higher	Lower	Lowest

16. Is It a Performance Feature?

No.

Module Import Declarations primarily affect **source-code ergonomics**.

They don't fundamentally change:

CPU usage
Memory usage
GC behavior
JIT optimization
Application throughput
Application latency

So:

Performance area	Effect
Startup performance	✗ No meaningful improvement
Runtime performance	✗ No meaningful improvement
Memory consumption	✗ No meaningful improvement

Compilation convenience	✓ Yes
Source-code size	✓ Can reduce
Developer productivity	✓ Yes
Code readability for small programs	✓ Often improves

17. Module Import Declaration and Java Modules

Don't confuse:

```
import module java.base;
```

with:

```
module my.application {
    requires java.sql;
}
```

They serve different purposes.

Concept	Purpose
module-info.java	Defines module dependencies/exports
requires	Declares a module dependency
exports	Makes a package available to other modules
import module	Makes accessible types from a module available by simple name in source code

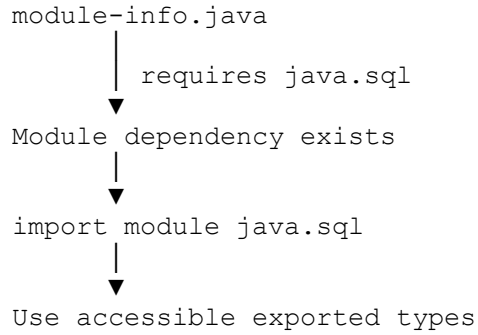
For example:

```
module my.application {
    requires java.sql;
}
```

and:

```
import module java.sql;
```

can conceptually work together:



The import declaration does not replace requires.

18. Overall Assessment

Category	Rating	Assessment
Developer productivity	★ ★ ★ ★ ★	Excellent
Boilerplate reduction	★ ★ ★ ★ ★	Excellent
Beginner friendliness	★ ★ ★ ★ ★	Excellent
Small applications	★ ★ ★ ★ ★	Excellent
Educational code	★ ★ ★ ★ ★	Excellent
Prototyping	★ ★ ★ ★ ★	Excellent
Production applications	★ ★ ★ ★	Useful selectively
Library development	★ ★ ★	Explicit imports may be preferable
Large enterprise code	★ ★ ★	Depends on project conventions
Runtime performance	★	Not a performance feature
Dependency clarity	★ ★ ★	Less precise than explicit imports

19. Final Decision Table

Scenario	Recommendation
Java tutorial	☐ Use Module Import
Learning Java	☐ Use Module Import
Small standalone application	☐ Use Module Import
Coding demonstration	☐ Use Module Import
Prototype	☐ Use Module Import
Small CLI utility	☐ Use Module Import
Many classes from the same module	☐ Use Module Import
Large enterprise application	☐ Use selectively
Public library/API	☐ Prefer explicit imports
Many modules with conflicting type names	☐ Prefer explicit imports
Strict dependency visibility required	☐ Prefer explicit imports
Runtime performance optimization	☐ Not relevant

Bottom line

Module Import Declarations are primarily a productivity and readability feature, not a performance feature.

Small / educational / prototype

↓
import module java.base;
↓
★ Excellent choice

Large production/library code

↓
Explicit imports
↓
★ More precise

The best practical rule is:

Use module imports when convenience and concise source code are more valuable than explicitly documenting every imported type; use individual imports when precision and dependency visibility matter most.

Java 25 — Compact Source Files and Instance Main Methods

Aspect	Details
Feature	Compact Source Files and Instance Main Methods
UniqueClassName	CompactSourceFilesInstanceMain
JEP	JEP 512
Java Version	25
Year	2025
Status	Final
Primary goal	Make simple Java programs easier to write and learn
Main concepts	Compact source files + instance main methods
Traditional entry point	public static void main(String[] args)
New simple entry point	void main() or other supported instance main forms
Typical target	Beginners, small programs, scripts, educational examples, prototypes

Java 25 finalized the feature that allows a simple Java program to omit the surrounding class declaration and use an **instance main method**, substantially reducing the amount of ceremony required for a basic program.

1. What Problem Does It Solve?

Traditional Java requires a beginner to write:

```
public class HelloWorld {
```

```

    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}

```

For a tiny program, there is a lot of syntax that has nothing to do with the actual lesson:

```

public class HelloWorld
    |
    |— required wrapper

public static void main(String[] args)
    |
    |— static
    |— public
    |— String[] args

```

Java 25 allows a much simpler form:

```

void main() {
    System.out.println("Hello, World!");
}

```

The source file itself can serve as the **compact source file**.

2. Traditional vs Compact Source File

Traditional Java	Java 25 Compact Source File
Requires explicit class	Class can be implicit
Requires public class	✗ Not required
Usually uses static main	Instance main supported
public static void main(String[] args)	void main()
More boilerplate	Much less boilerplate
Excellent for production	Excellent for small/simple programs
Beginner must learn class structure first	Beginner can start with executable statements

Traditional

```

public class Calculator {

```

```

public static void main(String[] args) {
    int a = 10;
    int b = 20;

    System.out.println(a + b);
}
}

```

Compact

```

void main() {
    int a = 10;
    int b = 20;

    System.out.println(a + b);
}

```

The second version lets the learner immediately focus on:

```

variables
↓
expressions
↓
statements
↓
output

```

3. Advantages

Advantage	Explanation	Real-world benefit
Much less boilerplate	No explicit surrounding class is required	Smaller source files
Beginner-friendly	Students can start with executable code	Easier introduction to Java
Simple entry point	void main() is much easier to understand	Reduces initial cognitive load
Faster experimentation	Small programs can be created quickly	Excellent for experimentation
Better educational examples	Examples can focus on the concept being taught	Cleaner tutorials

Good for command-line utilities	Tiny utilities don't need elaborate class structure	Faster development
Good for prototyping	Ideas can be tested quickly	Less ceremony
Instance state becomes natural	An instance main can work with instance fields/methods	Useful when teaching object-oriented concepts
Works with other modern Java simplifications	Can be combined with features such as module imports	Very concise beginner programs
Gradual learning path	Students can begin simply and later learn explicit classes	Smooth transition to conventional Java
Still uses Java's type system	It isn't a separate scripting language	Familiar Java semantics
Useful for small scripts	Simple one-file programs become more practical	Convenient for automation/tasks

4. Disadvantages

Disadvantage	Explanation	Impact
Can hide Java's class model	Beginners may not initially see that Java programs ultimately involve classes/types	Must eventually learn conventional classes
Not ideal for large applications	Large systems naturally need explicit classes and organization	Compact source files lose their advantage
Less explicit structure	The generated/implicit class isn't visible in the source	Can make architecture less obvious
Potential transition cost	Beginners eventually need to understand normal class declarations	Learning path must progress
Not a performance feature	It mainly changes source-code ergonomics	Doesn't improve runtime performance

Limited usefulness for frameworks	Enterprise frameworks generally expect explicit types and structures	Conventional classes remain preferable
Can encourage overly large source files	Beginners may keep adding code to one compact source file	Maintainability can suffer
Less suitable for reusable APIs	Libraries need explicit public types	Normal class/interface declarations are better
Tooling/conventions	Some organizational conventions may prefer explicit classes	Team standards may discourage it
Not a replacement for OOP	It simplifies the entry point, not the entire Java type system	Classes/interfaces still matter for serious applications

5. When to Use

Situation	Use?	Reason
Learning Java	✓ Excellent	Removes unnecessary initial ceremony
Hello World	✓ Excellent	Very small program
Teaching variables	✓ Excellent	Focus stays on variables
Teaching loops	✓ Excellent	Removes unrelated class boilerplate
Teaching conditionals	✓ Excellent	Cleaner examples
Teaching methods	✓ Excellent	Easy experimentation
Small CLI utility	✓ Yes	Minimal source structure
Coding exercise	✓ Yes	Short and focused
Prototype	✓ Yes	Rapid experimentation
One-off automation	✓ Yes	Convenient
Small algorithm experiment	✓ Yes	Less setup

Small educational project	✓ Yes	Clear and concise
Large enterprise application	□ Usually no	Explicit architecture is preferable
Public library	✗ Usually no	Explicit types are important
Framework-heavy application	✗ Usually no	Frameworks commonly rely on explicit classes
Large reusable component	✗ No	Normal class/module organization is better

6. When Not to Use

Scenario	Why not?	Better approach
Large business application	Too much implicit structure	Explicit classes/packages/modules
REST API with many components	Controllers/services/repositories need explicit types	Conventional class architecture
Spring-based application	Framework components need explicit classes	Normal Java classes
Public library	API types should be explicit	Explicit public classes/interfaces
Complex domain model	Many domain objects require clear boundaries	Explicit classes/records/interfaces
Large team project	Explicit structure helps collaboration	Conventional Java source
Complex inheritance hierarchy	Implicit class isn't useful	Explicit classes/interfaces
Framework extension point	Framework expects identifiable types	Explicit implementation classes
Performance optimization	No meaningful runtime benefit	Optimize algorithms/JIT/GC/I/O
Long-lived production code	Explicit organization improves maintainability	Conventional source structure

--	--	--

7. Real-Time Use Case — Hello World

Traditional

```
public class HelloWorld {  
  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

Java 25

```
void main() {  
    System.out.println("Hello World");  
}
```

Benefit

```
Traditional  
Class + main declaration + code  
    ↓  
More syntax  
  
Compact  
Code  
    ↓  
Immediate result
```

This is arguably the most obvious use case.

8. Real-Time Use Case — Calculator

A small calculator:

```
void main() {  
  
    int price = 100;  
    int quantity = 5;  
  
    int total = price * quantity;  
  
    System.out.println("Total = " + total);  
}
```

```
}
```

Instead of:

```
public class Calculator {  
  
    public static void main(String[] args) {  
  
        int price = 100;  
        int quantity = 5;  
  
        int total = price * quantity;  
  
        System.out.println("Total = " + total);  
    }  
}
```

For an introductory programming exercise, the compact form is much easier to read.

9. Real-Time Use Case — File Processing Utility

A small utility could look like:

```
import module java.base;  
  
void main() throws Exception {  
  
    var path = Path.of("orders.txt");  
  
    var lines = Files.readAllLines(path);  
  
    System.out.println("Orders: " + lines.size());  
}
```

This is especially attractive when combined with **Module Import Declarations**.

The resulting source is very close to a scripting-language experience while remaining Java.

10. Real-Time Use Case — Data Processing

Suppose you want to demonstrate filtering:

```
import module java.base;  
  
void main() {
```

```

var numbers = List.of(10, 15, 20, 25, 30);

for (var number : numbers) {
    if (number > 20) {
        System.out.println(number);
    }
}
}

```

The student can focus entirely on:

List
for-each
if
comparison

rather than class boilerplate.

11. Real-Time Use Case — Instance State

One interesting aspect is that the main method can be an **instance method**.

For example:

```

int counter = 0;

void main() {

    counter++;

    printCounter();
}

void printCounter() {
    System.out.println("Counter = " + counter);
}

```

Conceptually:

```

Implicit program class
├── counter
├── main()
└── printCounter()

```

This can make it easier to demonstrate the relationship between:

- fields
- methods
- object state
- instance behavior

without initially requiring students to write a class declaration.

12. Real-Time Use Case — Small Algorithm

For example, demonstrating factorial:

```
void main() {  
  
    int number = 5;  
    long result = 1;  
  
    for (int i = 1; i <= number; i++) {  
        result *= i;  
    }  
  
    System.out.println("Factorial = " + result);  
}
```

This is ideal for:

- coding exercises
 - classroom demonstrations
 - algorithm experimentation
 - interview practice
 - quick testing
-

13. Real-Time Use Case — Quick API Experiment

Suppose you want to test the Java time API:

```
import module java.base;  
  
void main() {  
  
    var today = LocalDate.now();  
    var tomorrow = today.plusDays(1);  
  
    System.out.println("Today: " + today);  
}
```

```
    System.out.println("Tomorrow: " + tomorrow);  
}
```

Instead of creating:

```
public class DateDemo {  
    public static void main(String[] args) {  
        ...  
    }  
}
```

the developer can concentrate on the API being tested.

14. Compact Source Files + Module Import Declarations

These two Java 25 features complement each other particularly well.

Without the two features

```
import java.time.LocalDate;  
import java.util.ArrayList;  
import java.util.List;  
  
public class Demo {  
  
    public static void main(String[] args) {  
        List<String> names = new ArrayList<>();  
  
        names.add("Java");  
  
        System.out.println(LocalDate.now());  
        System.out.println(names);  
    }  
}
```

With both

```
import module java.base;  
  
void main() {  
  
    List<String> names = new ArrayList<>();  
  
    names.add("Java");  
  
    System.out.println(LocalDate.now());  
    System.out.println(names);  
}
```

}

Result

Aspect	Traditional	Java 25 concise style
Explicit class	Required	✗
public static main	Required traditionally	✗
Individual imports	Several	Potentially one module import
Boilerplate	Higher	Much lower
Beginner readability	Moderate	High

15. Compact Source Files vs Traditional Classes

Criterion	Compact Source File	Explicit Class
Small programs	★ ★ ★ ★ ★	★ ★ ★
Beginner learning	★ ★ ★ ★ ★	★ ★ ★
Tutorials	★ ★ ★ ★ ★	★ ★ ★ ★
Prototypes	★ ★ ★ ★ ★	★ ★ ★
One-off utilities	★ ★ ★ ★ ★	★ ★ ★ ★
Large applications	★ ★	★ ★ ★ ★ ★
Reusable libraries	★	★ ★ ★ ★ ★
Explicit architecture	★ ★	★ ★ ★ ★ ★
Framework integration	★ ★	★ ★ ★ ★ ★
OOP-heavy design	★ ★	★ ★ ★ ★ ★
API design	★	★ ★ ★ ★ ★
Boilerplate	★ ★ ★ ★ ★	★ ★ ★

16. Alternate Features / Approaches

There isn't a single direct replacement. The best alternative depends on the type of program.

Requirement	Alternative	When preferable
Conventional Java application	Explicit class + main	Production applications
Reusable component	Explicit class/interface/record	Library/API development
Small example	Compact source file	Educational/demo code
Quick command-line program	Compact source file	Small one-off utilities
Functional-style small operation	Lambda / method reference	When behavior can be represented as a function
Shell-style scripting	Java launcher + compact source	Quick Java scripts
Multi-class application	Normal class-based structure	Larger programs
Enterprise application	Explicit classes/packages/modules	Maintainability and architecture

17. Compact Source Files vs JShell

Another useful comparison is with **JShell**.

Aspect	Compact Source Files	JShell
Persistent source file	✓	☐ Session-oriented
Compile/run application	✓	☒ Primarily interactive
Small programs	★★★★★	★★★★★
Interactive experimentation	★★★	★★★★★
Educational use	★★★★★	★★★★★

Version-controlled program	★ ★ ★ ★ ★	★ ★
Multi-method program	✓	Limited/different model
Production application	☐ Small apps	✗
REPL experience	✗	✓

Rule:

Interactive experimentation → JShell

Small executable source file → Compact Source File

Production application → Explicit classes

18. Important Clarification — Is It a Performance Feature?

No.

Compact Source Files and Instance Main Methods primarily improve **developer experience and source-code simplicity**.

Performance area	Expected benefit
Runtime speed	✗ No meaningful direct improvement
CPU usage	✗ No
Memory usage	✗ No
GC performance	✗ No
JIT optimization	✗ Not inherently
Startup time	✗ Not the purpose
Source-code size	✓ Smaller
Development speed	✓ Higher
Beginner accessibility	✓ Much higher
Experimentation speed	✓ Higher

The JVM still executes Java bytecode; the feature mainly changes how the source program is expressed.

19. Overall Assessment

Category	Rating	Assessment
Beginner friendliness	★ ★ ★ ★ ★	Excellent
Boilerplate reduction	★ ★ ★ ★ ★	Excellent
Educational use	★ ★ ★ ★ ★	Excellent
Small programs	★ ★ ★ ★ ★	Excellent
Prototyping	★ ★ ★ ★ ★	Excellent
CLI utilities	★ ★ ★ ★ ★	Excellent
Quick experiments	★ ★ ★ ★ ★	Excellent
Production applications	★ ★ ★	Selective use
Enterprise systems	★ ★	Explicit classes usually better
Library development	★	Explicit types strongly preferred
API design	★	Not its purpose
Runtime performance	★	Not a performance feature

20. Final Decision Table

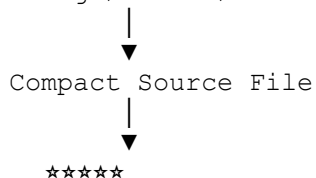
Real-Time Scenario	Recommendation
Hello World	☐ Use compact source file
Teaching Java basics	☐ Use compact source file
Teaching loops/conditions	☐ Use compact source file
Coding exercises	☐ Use compact source file
Small CLI tool	☐ Use compact source file

Prototype	☐ Use compact source file
One-off automation	☐ Use compact source file
API experimentation	☐ Use compact source file
Small personal utility	☐ Use compact source file
Large REST application	☐ Use explicit classes
Spring/enterprise application	☐ Use explicit classes
Public Java library	☐ Use explicit classes/interfaces
Complex domain model	☐ Use explicit classes/records/interfaces
Large team project	☐ Generally explicit classes
Runtime performance optimization	☐ Feature is irrelevant

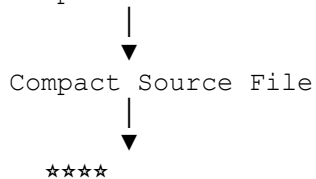
Bottom line

Compact Source Files and Instance Main Methods are primarily a simplicity feature. They make Java much more approachable for small executable programs without changing Java's underlying object-oriented and type-safe model.

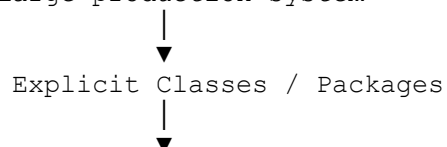
Learning / Demo / Prototype



Small production utility



Large production system



Best rule:

Use compact source files for small, focused, executable programs; use explicit classes when structure, reuse, APIs, framework integration, or long-term maintainability becomes important.

Java 25 — Flexible Constructor Bodies

Aspect	Details
Feature	Flexible Constructor Bodies
UniqueClassName	FlexibleConstructorBodies
JEP	JEP 513
Java Version	25
Year	2025
Status	Final
Main purpose	Allow statements in a constructor before an explicit <code>super(...)</code> or <code>this(...)</code> constructor invocation, subject to strict restrictions
Key benefit	Allows safe validation, normalization, temporary computation, and argument preparation before invoking the superclass constructor
Earlier status	Preview in Java 22, Java 23, and Java 24
Important restriction	Statements before <code>super(...)</code> / <code>this(...)</code> cannot access the object being constructed in unsafe ways, such as accessing instance fields or invoking instance methods

1. What Are Flexible Constructor Bodies?

Traditionally, Java requires an explicit constructor invocation such as:

```
super(...);
```

or

```
this(...);
```

to occur before ordinary constructor statements.

For example:

```
class Employee extends Person {  
  
    Employee(String name) {  
        super(name);  
        System.out.println("Employee created");  
    }  
}
```

Before Java 25, you couldn't generally do arbitrary preparation before `super(...)`.

Java 25 allows controlled statements before the constructor invocation:

```
class Employee extends Person {  
  
    Employee(String name) {  
  
        String normalizedName = name.trim();  
  
        super(normalizedName);  
    }  
}
```

The important idea is:

Traditional constructor

```
constructor()  
  |  
  ▼  
super(...)  
  |  
  ▼  
constructor body
```

Flexible constructor

```
constructor()  
  |  
  ▼  
validate / normalize / calculate  
  |  
  ▼
```

```
super(...)  
  |  
  ▼  
constructor body
```

This is especially useful when the arguments required by the superclass need to be **validated, transformed, or computed first**.

2. Why Was This Needed?

Consider:

```
class Person {  
    Person(String name) {  
        ...  
    }  
}  
  
class Employee extends Person {  
    Employee(String name) {  
        super(name.trim());  
    }  
}
```

This works because the expression passed to `super()` can perform some computation.

But imagine more complicated preparation:

```
class Employee extends Person {  
    Employee(String name, String department) {  
        String normalizedName = normalize(name);  
        String normalizedDepartment = normalize(department);  
        super(normalizedName, normalizedDepartment);  
    }  
}
```

The ability to write these preparation statements directly before `super(...)` makes the constructor easier to express.

3. Advantages

Advantage	Explanation	Real-world benefit
Pre-super preparation	Allows certain statements before <code>super(...)</code> or <code>this(...)</code>	Easier constructor argument preparation
Validation before delegation	Constructor arguments can be checked before superclass construction	Cleaner validation logic
Normalization	Input can be trimmed, canonicalized, or transformed	Better data consistency
Temporary variables	Complex calculations can be assigned to local variables	Improves readability
Better inheritance design	Subclass can prepare values needed by superclass	Cleaner subclass constructors
Reduces awkward helper constructors	Some preparation no longer needs to be moved into another constructor	Less boilerplate
Improves readability	Complex expressions don't have to be embedded inside <code>super(...)</code>	Easier maintenance
Useful with immutable objects	Values can be normalized before the superclass initializes state	Better object construction
Supports defensive validation	Invalid constructor input can be rejected early	Prevents invalid object creation
More expressive constructors	Constructor logic can follow a natural sequence	Easier to understand
Works with <code>this(...)</code> as well	Preparation can occur before constructor delegation	More flexible constructor chains

4. Disadvantages

Disadvantage	Explanation	Impact
Object is not fully initialized	The superclass constructor has not run yet	Strong restrictions remain
Cannot freely access instance state	Instance fields/methods may not be safely available	Limits what can be done before <code>super()</code>
More constructor complexity	Developers can now put more logic into constructors	Can lead to oversized constructors
Potentially confusing for beginners	Constructor execution order becomes more subtle	Higher learning curve
Easy to misuse	Developers may put too much validation/business logic before <code>super()</code>	Reduced maintainability
Not a replacement for factory methods	Complex object creation may still be better represented by a factory	Architectural complexity can remain
Not a performance feature	The feature primarily improves source expressiveness	No automatic runtime speedup
Inheritance remains complex	Flexible constructor bodies don't eliminate superclass initialization rules	Constructor design still requires care
Static/helper methods may be needed	Some operations must avoid depending on the partially initialized object	Additional design considerations
Limited use cases	Most ordinary constructors don't actually need statements before <code>super()</code>	Don't use it simply because it's available

5. When to Use

Situation	Use?	Why
Normalize constructor arguments	✓ Yes	Cleanly prepare values before superclass construction

Validate arguments	✓ Yes	Reject invalid input early
Calculate superclass parameters	✓ Yes	Avoid complicated super(...) expressions
Parse input before superclass initialization	✓ Yes	Makes constructor logic clearer
Convert units/formats	✓ Yes	Prepare values before delegation
Prepare multiple local variables	✓ Yes	Improves readability
Subclass constructor needs transformed values	✓ Yes	Strong use case
Constructor delegation with this(...) needs preparation	✓ Yes	Flexible construction flow
Simple constructor	☐ Usually unnecessary	Normal constructor is clearer
Complex business logic	☐ Prefer factory/service	Constructors should remain focused

6. When Not to Use

Scenario	Don't use?	Why	Better approach
Simple field assignment	✓	No benefit	Normal constructor
No preprocessing required	✓	Adds unnecessary complexity	super(...) directly
Large business rules	✓	Constructor becomes difficult to maintain	Factory/service
Database access	✓	Constructors shouldn't normally perform I/O	Factory/service/repository
Network calls	✓	Expensive and	Service/factory

		failure-prone	
Complex object creation	✓	Better abstraction exists	Factory method/builder
Operations requiring instance state	✓	Object isn't safely initialized before superclass construction	Move logic after super()
Performance optimization	✓	Not intended for runtime optimization	Optimize actual bottleneck
Highly reusable API	□	Constructor behavior should remain predictable	Explicit factory/builder APIs

7. Real-Time Use Case — Input Normalization

Suppose a superclass expects a normalized customer name.

```
class Person {
    Person(String name) {
        System.out.println("Person: " + name);
    }
}
```

```
class Customer extends Person {
    Customer(String name) {
        String normalized = name.trim();
        super(normalized);
    }
}
```

Conceptually:

```
"  Anurag  "
  |
  ▼
trim()
  |
  ▼
```

```
"Anurag"
  |
  ▼
super(...)
```

Benefit

Without flexible constructor bodies, you may have to put increasingly complex expressions into the constructor invocation or move preprocessing elsewhere.

8. Real-Time Use Case — Validation

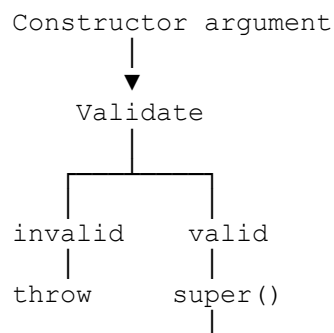
Imagine a subclass that needs to ensure an ID is valid before passing it to the superclass.

```
class Account {
    Account(String accountId) {
        ...
    }
}

class BankAccount extends Account {
    BankAccount(String accountId) {
        if (accountId == null || accountId.isBlank()) {
            throw new IllegalArgumentException(
                "Account ID cannot be empty"
            );
        }

        super(accountId);
    }
}
```

Conceptually:



▼
Object creation

This can make the intent clearer than hiding validation inside a complex expression.

9. Real-Time Use Case — Multiple Calculations

Suppose a superclass expects a calculated value.

```
class Product {  
    Product(String code, double price) {  
        ...  
    }  
}  
  
class DiscountedProduct extends Product {  
    DiscountedProduct(  
        String code,  
        double price,  
        double discount) {  
        double discountedPrice =  
            price - (price * discount);  
        super(code, discountedPrice);  
    }  
}
```

The local variable makes the transformation easy to understand.

10. Real-Time Use Case — Unit Conversion

Suppose a superclass expects meters while the subclass API accepts centimeters.

```
class Measurement {  
    Measurement(double meters) {  
        ...  
    }  
}  
  
class RoomMeasurement extends Measurement {
```

```

RoomMeasurement(double centimeters) {
    double meters = centimeters / 100.0;
    super(meters);
}

```

Flow:

```

centimeters
  ↓
/ 100
  ↓
meters
  ↓
super(meters)

```

This is a clean example of **argument preparation before superclass initialization**.

11. Real-Time Use Case — Parsing

Consider a superclass that expects an integer:

```

class User {
    User(int userId) {
        ...
    }
}

class DatabaseUser extends User {
    DatabaseUser(String userIdText) {
        int userId = Integer.parseInt(userIdText);
        super(userId);
    }
}

```

The subclass API accepts a string, but the superclass requires an integer.

Flexible constructor bodies allow the conversion to be expressed naturally.

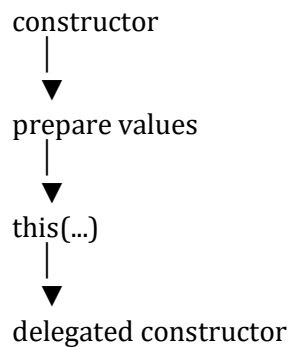
12. Real-Time Use Case — Constructor Delegation

Flexible constructor bodies can also be used before `this(...)`.

Conceptually:

```
class Report {  
    Report(String name, int size) {  
        ...  
    }  
  
    Report(String name) {  
        int defaultSize = calculateDefaultSize();  
        this(name, defaultSize);  
    }  
}
```

The important concept is:



However, the same restrictions regarding the partially constructed object apply.

13. Important Restriction — Partially Constructed Object

This is one of the most important things to understand.

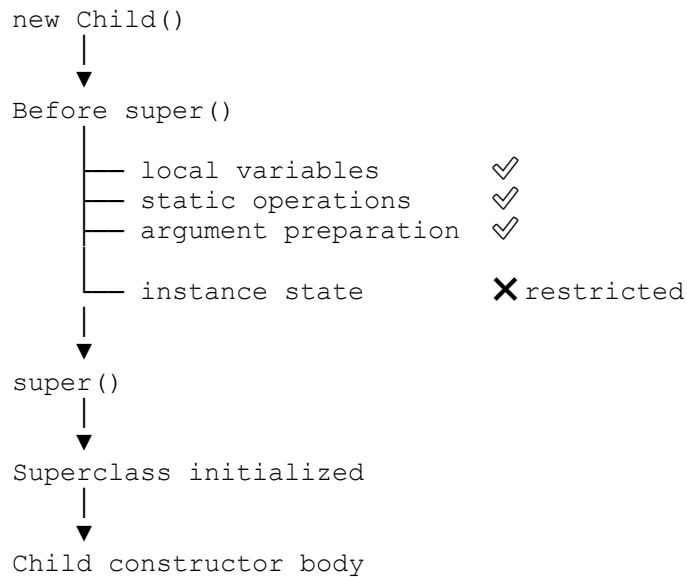
Before:

```
super(...);
```

the superclass portion of the object hasn't been initialized.

Therefore, you shouldn't think of the object as fully available.

Conceptually:



This restriction exists to preserve Java's object-initialization safety.

14. Flexible Constructor Bodies vs Factory Methods

A factory method can often be a better alternative when preparation becomes complicated.

Flexible constructor

```
class Employee extends Person {  
    Employee(String name) {  
        String normalized = normalize(name);  
        super(normalized);  
    }  
}
```

Factory

```
class Employee extends Person {
```

```

private Employee(String name) {
    super(name);
}

static Employee create(String name) {
    String normalized = normalize(name);
    return new Employee(normalized);
}
}

```

Criterion	Flexible Constructor	Factory Method
Simple preprocessing	★ ★ ★ ★ ★	★ ★ ★
Complex creation logic	★ ★	★ ★ ★ ★ ★
Multiple creation strategies	★ ★	★ ★ ★ ★ ★
Validation	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Readability	★ ★ ★ ★	★ ★ ★ ★ ★
Constructor simplicity	★ ★ ★ ★	★ ★ ★ ★ ★
Object creation abstraction	★ ★ ★	★ ★ ★ ★ ★

15. Alternate Features / Approaches

There is no single direct replacement. The appropriate alternative depends on the requirement.

Requirement	Alternative	When preferable
Simple initialization	Traditional constructor	Most ordinary classes
Complex preprocessing	Static factory method	When creation logic is substantial
Many optional parameters	Builder pattern	Complex object configuration
Input normalization	Helper/static method	When transformation is reusable
Validation	Factory + validation	Complex validation rules
Multiple construction	Factory methods	Different ways to create the object

strategies		
Immutable complex object	Builder	Many fields/configuration options
Dependency-heavy creation	Factory/service/DI container	Enterprise applications
Simple subclass delegation	super(...) directly	No preprocessing needed
Constructor chaining	this(...)	Reusing initialization logic

16. Flexible Constructor Bodies vs Traditional Constructor

Feature	Traditional Constructor	Flexible Constructor Bodies
Statements before super()	✗	✓
Argument preprocessing	Limited	Better
Local variables before super()	✗	✓ within restrictions
Validation before super()	Limited	Better
Simple constructors	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Complex subclass construction	★ ★ ★	★ ★ ★ ★ ★
Beginner simplicity	★ ★ ★ ★ ★	★ ★ ★
Runtime performance	Same	Same
Constructor expressiveness	★ ★ ★	★ ★ ★ ★ ★

17. Flexible Constructor Bodies vs Builder

Scenario	Flexible Constructor	Builder
----------	----------------------	---------

2-3 constructor parameters	★ ★ ★ ★ ★	★ ★
Normalize input	★ ★ ★ ★ ★	★ ★ ★
Simple validation	★ ★ ★ ★ ★	★ ★ ★
10+ optional parameters	★ ★	★ ★ ★ ★ ★
Immutable complex object	★ ★ ★	★ ★ ★ ★ ★
Multiple configurations	★ ★	★ ★ ★ ★ ★
Inheritance preprocessing	★ ★ ★ ★ ★	★ ★ ★
Readability for complex creation	★ ★ ★	★ ★ ★ ★ ★

18. Is It a Performance Feature?

No.

Flexible Constructor Bodies primarily improve **expressiveness and maintainability**.

Performance area	Direct benefit
CPU performance	✗ No
Memory usage	✗ No
GC performance	✗ No
JIT optimization	✗ No direct benefit
Startup time	✗ No
Constructor readability	✓ Yes
Developer productivity	✓ Yes
Code maintainability	✓ Yes
Inheritance flexibility	✓ Yes

Any performance difference would come from the actual code you put into the constructor—not from the feature itself.

19. Overall Assessment

Category	Rating	Assessment
Constructor flexibility	★ ★ ★ ★ ★	Excellent
Argument preprocessing	★ ★ ★ ★ ★	Excellent
Validation	★ ★ ★ ★ ★	Very useful
Normalization	★ ★ ★ ★ ★	Very useful
Inheritance	★ ★ ★ ★ ★	Strong use case
Readability	★ ★ ★ ★	Better for suitable cases
Beginner friendliness	★ ★ ★	Requires understanding initialization order
Complex object creation	★ ★ ★	Factory/builder may be better
Large business logic	★ ★	Avoid
Runtime performance	★	Not a performance feature
API design	★ ★ ★ ★	Useful when applied selectively

20. Final Decision Table

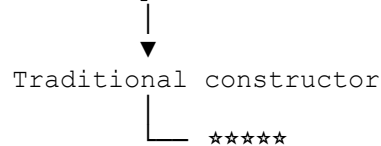
Real-Time Scenario	Recommendation
Normalize subclass constructor arguments	☐ Use
Validate arguments before <code>super()</code>	☐ Use
Calculate superclass parameters	☐ Use
Convert units before superclass initialization	☐ Use

Parse constructor input	☐ Use
Prepare several local values	☐ Use
Simple super(...) call	☐ Traditional constructor is better
Simple field initialization	☐ Traditional constructor
Complex business logic	☐ Use factory/service
Many optional parameters	☐ Consider Builder
Database/network operations	☐ Avoid in constructor
Complex object creation	☐ Factory/Builder
Runtime performance optimization	☐ Not relevant

Bottom line

Flexible Constructor Bodies are most valuable when a subclass needs to prepare, validate, normalize, or calculate constructor arguments before invoking super(...) or this(...).

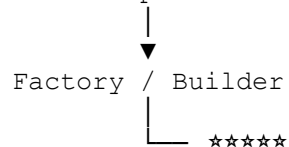
Need simple initialization?



Need preprocessing before super()/this()?



Need complex creation logic?



Best rule: Use Flexible Constructor Bodies for **small, local, constructor-specific preparation**. If the preparation becomes complex business logic, move it into a **factory method, builder, or dedicated service** rather than turning the constructor into a mini-business layer.

Java 25 — Ahead-of-Time Command-Line Ergonomics

Aspect	Details
Feature	Ahead-of-Time (AOT) Command-Line Ergonomics
UniqueClassName	AOTCommandLineErgonomics
JEP	JEP 514
Java Version	25
Year	2025
Status	Final
Project	Project Leyden
Primary goal	Simplify creation and use of AOT caches
Main benefit	Better startup/warm-up with substantially simpler AOT commands
Main area	JVM startup and deployment
Runtime API?	No — primarily JVM/launcher behavior
Works with	AOT cache, class loading/linking, CDS-related infrastructure

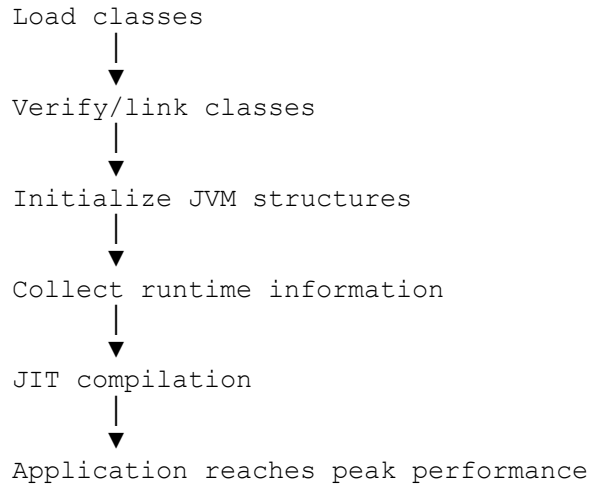
JEP 514 is part of Project Leyden's effort to improve Java application **startup time, time to peak performance, and footprint**. It builds on JEP 483, which introduced AOT class loading and linking in Java 24. ([Oracle Docs](#))

1. What Is Ahead-of-Time Command-Line Ergonomics?

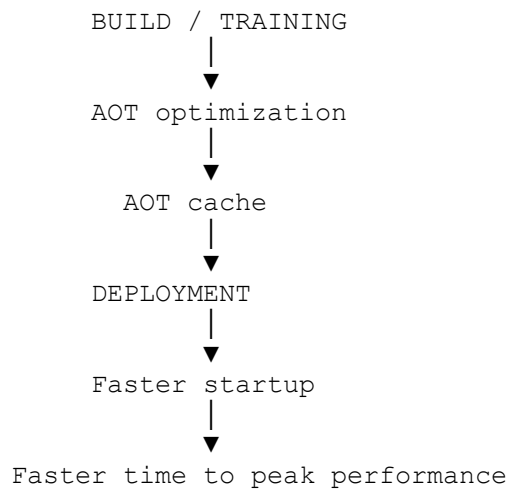
Normally, Java applications perform a lot of work at startup:

Application starts





AOT caching moves some useful work earlier:



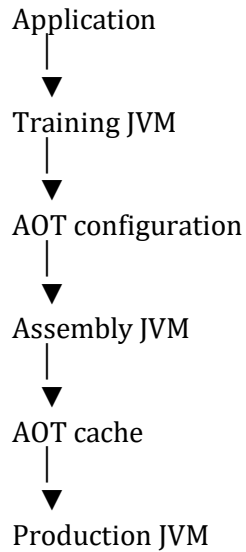
JDK 25 simplifies the commands required to create these AOT caches. Oracle describes the feature as reducing the complexity of commands needed for common AOT-cache use cases. ([Oracle Docs](#))

2. What Was Difficult Before?

A traditional AOT-cache workflow could involve several phases:

1. Training
↓
2. Record configuration
↓
3. Assemble cache
↓
4. Deploy cache

Conceptually:



Java 25 makes common workflows considerably simpler.

For example, JEP 514 supports a one-command cache-creation workflow using:

```
java -cp HelloWorld.jar \  
-XX:AOTMode=record \  
-XX:AOTCacheOutput=foo.aot \  
HelloWorld
```

The JVM can perform the recording and launch the assembly process automatically. ([OpenJDK Mail](#))

3. Important AOT Modes

Java 25 provides the following AOT modes:

AOTMode	Purpose
auto	Automatically determine the appropriate behavior
off	Disable AOT
record	Run the application and collect information for AOT creation
create	Create the AOT cache from recorded configuration

on	Use an existing AOT cache
-----------	---------------------------

auto is the default. In production, this is particularly useful because the application can continue normally if an AOT cache cannot be used. ([Oracle Docs](#))

4. Advantages

Advantage	Explanation	Real-world benefit
Simpler AOT workflow	Reduces the number of commands required for common cases	Easier deployment
Faster startup	AOT caches can avoid repeating some startup work	Better application responsiveness
Better warm-up	Precomputed artifacts reduce work during initial execution	Faster time to useful performance
One-command cache creation	AOTCacheOutput can simplify common cache-generation workflows	Easier CI/CD integration
Automatic mode selection	AOTMode=auto can select record/on/off behavior	Less manual JVM configuration
Production-friendly fallback	In auto, failure to load the cache does not normally prevent application execution	Safer deployments
Build/deployment automation	AOT cache generation can be incorporated into build pipelines	Repeatable deployments
Useful for short-lived applications	Startup overhead matters more when applications don't run for hours	Better efficiency
Works at JVM level	Applications don't need special AOT APIs in their Java code	Minimal application changes
Builds on JEP 483	Uses AOT class-	Incremental improvement

	loading/linking infrastructure introduced in Java 24	
Useful for containers	Pre-created cache can be packaged with the application image	Faster container startup
Complements JEP 515	AOT method profiling can further improve startup/warm-up	Stronger startup optimization

Oracle specifically notes that AOT caches can improve startup and warm-up performance, and that JDK 25 simplifies their creation. ([Oracle Docs](#))

5. Disadvantages

Disadvantage	Explanation	Impact
Additional build complexity	AOT caching introduces another build/deployment artifact	CI/CD becomes more involved
Cache is environment-specific	A cache is tied to the application, JDK release, OS and CPU architecture	Cannot freely copy caches between environments
Cache must be regenerated after important changes	Application/JDK/platform changes can invalidate the cache	Additional deployment work
Training workload matters	AOT optimization depends on representative application execution	Poor training can reduce benefits
Not useful for every application	Long-running applications may get little startup benefit	Benchmark before adopting
Extra storage	A .aot cache is another deployment artifact	Larger images/artifacts
Compatibility constraints	Some JVM options can prevent cache use	Deployment configuration matters
Not equivalent to native	Java still runs on the JVM	Don't confuse it with GraalVM

compilation		Native Image
Potential operational complexity	Multiple JVM/AOT settings can complicate troubleshooting	Requires operational knowledge
No guaranteed performance improvement	Benefit depends on application characteristics	Benchmark real workloads
Cache lifecycle management	Old caches need to be removed/rebuilt	More release-management work

An AOT cache is specific to the application, JDK release, OS, and CPU architecture. If those change, the cache needs to be recreated. ([Oracle Docs](#))

6. When to Use

Scenario	Use?	Why
Serverless Java function	☐ Excellent	Startup latency is important
Containerized microservice	☐ Very good	Faster container/application startup
CLI application	☐ Good	Startup can dominate total execution time
Frequently restarted service	☐ Good	Startup cost occurs repeatedly
Kubernetes workloads	☐ Good	Faster pod startup/readiness
Autoscaling applications	☐ Excellent	New instances can become useful sooner
Short-lived batch jobs	☐ Good	JVM startup represents a larger fraction of runtime
Development tools	☐ Potentially useful	Faster startup improves responsiveness
Large monolithic service running for days	☐ Benchmark first	Startup cost is amortized
CPU-bound long-running service	☐ Benchmark first	Steady-state performance may matter more

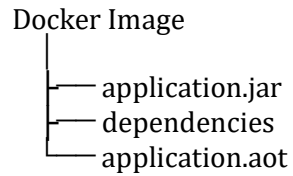
Highly dynamic deployment	☐ Careful	Cache compatibility becomes more important

7. When Not to Use

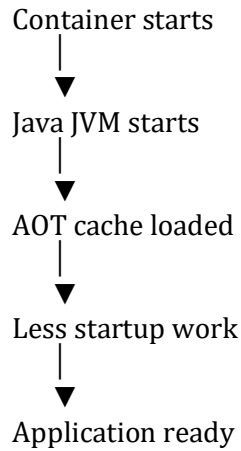
Scenario	Don't use?	Why	Better approach
Application runs continuously for weeks	☐ Usually unnecessary	Startup cost is tiny compared with lifetime	Normal JVM
Frequently changing classes	☐ Often	Cache needs frequent regeneration	Standard JVM/AOT only when stable
Highly heterogeneous infrastructure	☐	Cache is platform-specific	Generate per target environment
Tiny application already starts instantly	☐	Little measurable benefit	Standard JVM
Development with constantly changing code	☐ Usually	Cache invalidation reduces value	Normal JVM
Strictly portable deployment artifact	☐	AOT cache is environment-specific	Standard JAR/container
Trying to maximize peak throughput only	☐	AOT primarily targets startup/warm-up	Profile/JIT/GC tuning
Need native executable	☐	AOT cache isn't native compilation	GraalVM Native Image

8. Real-Time Use Case — Containerized Microservice

Suppose you deploy:



Startup:



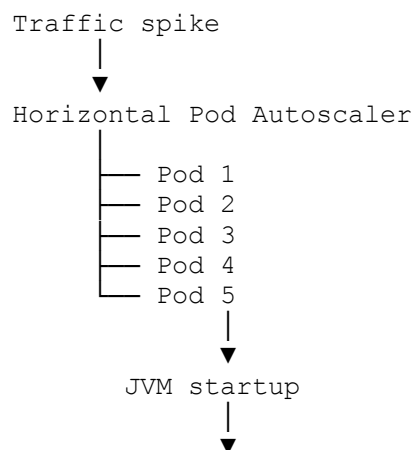
This can be particularly useful for services that frequently scale from:

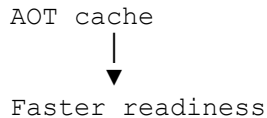
10 pods → 50 pods → 100 pods

because every new JVM instance has startup work.

9. Real-Time Use Case — Kubernetes Autoscaling

Consider:





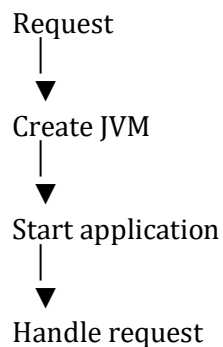
Why useful?

If a Java service normally takes significant time to become ready, faster startup can help the autoscaler respond to traffic increases.

Important: actual improvement must be measured for the specific application.

10. Real-Time Use Case — Serverless

Serverless environments often care about:



If JVM startup is:

100 ms

and actual work is:

20 ms

then startup dominates the request.

AOT optimizations can shift some work away from that critical path:

Before:

Startup 100 ms + Work 20 ms
↓
120 ms

With AOT:

Startup 60 ms + Work 20 ms
↓
80 ms

These numbers are **illustrative**, not guaranteed performance results.

11. Real-Time Use Case — CLI Application

Consider a Java command-line tool:

```
java -jar report-tool.jar
```

The application might:

```
Start JVM
↓
Load classes
↓
Initialize
↓
Read input
↓
Generate report
↓
Exit
```

If execution only takes a few hundred milliseconds, JVM startup can be a significant percentage of total execution time.

AOT caching can therefore be more valuable for this kind of application than for a server that runs continuously for days.

12. Real-Time Use Case — CI/CD Build Tool

Imagine a Java-based build utility:

```
CI Job
├── Start Java tool
├── Analyze files
├── Generate output
└── Exit
```

If the tool executes thousands of times across CI workers, reducing startup/warm-up overhead can accumulate into meaningful savings.

AOT can be considered when:

- the tool is stable,
 - the runtime/JDK environment is predictable,
 - startup is measurable,
 - the cache can be generated reliably.
-

13. Real-Time Use Case — Frequently Restarted Service

Some services restart because of:

- deployments,
- rolling updates,
- autoscaling,
- node maintenance,
- crash recovery.

Without AOT:

```
Deploy
↓
JVM startup
↓
Class loading/linking
↓
Warm-up
↓
Ready
```

With AOT:

```
Deploy
↓
JVM startup
↓
AOT cache
↓
Reduced startup/warm-up
↓
Ready
```

The benefit becomes more significant when the service is restarted frequently.

14. One-Command AOT Cache Creation

A simplified example:

```
java \  
-cp app.jar \  
-XX:AOTMode=record \  
-XX:AOTCacheOutput=app.aot \  
com.example.Main
```

The JDK can use this workflow to record the application and then launch another JVM to assemble the cache. ([OpenJDK Mail](#))

Production can then use:

```
java \  
-XX:AOTCache=app.aot \  
-cp app.jar \  
com.example.Main
```

Or, more generally, allow the JVM's default auto mode to determine whether an AOT cache can be used.

15. auto vs on

This distinction is particularly important.

Mode	Behavior	Recommended use
auto	Use cache when possible; otherwise continue normally	★ Production
on	Require/use AOT cache and fail if it cannot be loaded	Diagnostics/testing
off	Disable AOT	Troubleshooting/comparison
record	Training/recording phase	Cache creation
create	Assemble cache from configuration	Explicit build workflow

Oracle recommends auto for production because an incompatible or unavailable cache doesn't normally prevent the application from running. on is more appropriate as a fail-fast/debugging aid. ([Oracle Docs](#))

16. AOT Cache Compatibility

An AOT cache is tied to:

Application
+
JDK release
+
Operating system
+
CPU architecture

Therefore:

Build on:
JDK 25 + Linux x86-64
↓
app.aot
|
X
|
Run on:
JDK 25 + Windows ARM64

You should not assume that the same cache can be used.

Instead:

Linux x86-64 build
↓
Linux x86-64 AOT cache

Linux ARM64 build
↓
Linux ARM64 AOT cache

Windows x86-64 build
↓
Windows x86-64 AOT cache

This environment specificity is an important operational consideration. ([Oracle Docs](#))

17. AOT Command-Line Ergonomics vs Traditional CDS

Aspect	Traditional CDS	AOT Command-Line Ergonomics
Primary goal	Share class metadata	Broader AOT startup/warm-up optimization
Startup improvement	✓	✓
AOT cache	Limited/traditional archive concepts	✓
Training workflow	More limited	✓
Class loading/linking optimization	Some	✓
Simplified AOT creation	✗	✓
Best for	General JVM startup optimization	Applications where startup/warm-up matters

AOT caching builds on the JVM's existing class-sharing infrastructure rather than replacing every traditional JVM startup optimization. The AOT cache introduced in JDK 24 can contain classes and heap objects, with later releases adding more kinds of artifacts. ([Oracle Docs](#))

18. AOT Command-Line Ergonomics vs AOT Method Profiling

These are **different Java 25 features**.

Aspect	JEP 514 — AOT Command-Line Ergonomics	JEP 515 — AOT Method Profiling
Primary goal	Simplify AOT cache creation/use	Preserve method execution profiles
Main benefit	Easier AOT deployment	Faster JIT optimization at startup
Focus	AOT workflow/ergonomics	Method profiling
Startup benefit	✓	✓
Warm-up benefit	✓	Strong
Developer-facing	Mostly command-line	Mostly JVM internals
Replacement for other	✗	✗

Can complement each other	✓ Yes	✓ Yes

Oracle specifically describes JEP 515 as making previous method-execution profiles available when the JVM starts, allowing JIT compilation to use them immediately. ([Oracle Docs](#))

19. Alternate Features / Technologies

There isn't one exact replacement. Choose according to the performance problem.

Requirement	Alternative	When preferable
General startup optimization	CDS	When simpler class-sharing is sufficient
AOT class loading/linking	JEP 483	Foundation underlying JEP 514
Faster startup + warm-up	JEP 515 AOT Method Profiling	When method-profile reuse is valuable
Native executable	GraalVM Native Image	When JVM-free native deployment is required
JVM startup optimization	AOT cache	When staying on HotSpot/JVM is preferred
Runtime steady-state performance	JIT tuning/profiling	When startup isn't the bottleneck
Reduce application startup work	Lazy initialization	When application code itself dominates startup
Container startup optimization	AOT cache + optimized image	Predictable container environments

20. AOT Cache vs GraalVM Native Image

These are often confused.

Aspect	Java 25 AOT Cache	GraalVM Native Image
JVM required	✓	✗
Produces native executable	✗	✓
Keeps JVM runtime	✓	✗
Startup improvement	✓	★ ★ ★ ★ ★
Peak JIT optimization	✓ JVM-based	Different model
Java compatibility	High	Requires native-image compatibility considerations
Deployment model	JVM + JAR + cache	Native executable
Build complexity	Lower	Higher
Best for	JVM applications needing faster startup	Native/serverless/microservice deployments
Dynamic JVM behavior	Stronger	More constrained
Existing Java deployment model	Excellent fit	Requires different deployment model

So:

AOT cache ≠ native compilation.

AOT caching keeps the HotSpot JVM execution model while precomputing useful artifacts.

21. Is It a Runtime Performance Feature?

Yes, but specifically a startup/warm-up optimization.

It is not primarily designed to increase the maximum throughput of a long-running application.

Performance area	Expected impact
Startup time	★ ★ ★ ★ ★
Time to useful execution	★ ★ ★ ★ ★

Warm-up time	★ ★ ★ ★
Peak steady-state throughput	★ ★
Long-running CPU throughput	★ ★
Memory footprint	★ ★
Developer productivity	★ ★ ★ ★
Deployment simplicity	★ ★ ★ ★

The actual performance improvement depends heavily on the application and workload.

22. Overall Assessment

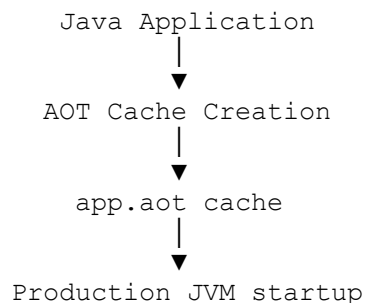
Category	Rating	Assessment
Startup optimization	★ ★ ★ ★ ★	Excellent
Warm-up optimization	★ ★ ★ ★	Very good
Serverless	★ ★ ★ ★ ★	Strong candidate
Containers	★ ★ ★ ★ ★	Strong candidate
Autoscaling	★ ★ ★ ★ ★	Very useful
CLI applications	★ ★ ★ ★	Often useful
CI/CD tools	★ ★ ★ ★	Potentially useful
Long-running applications	★ ★	Benchmark first
Operational simplicity	★ ★ ★ ★	Much better than earlier AOT workflows
Portability	★ ★	Cache is platform/JDK specific
Peak throughput	★ ★	Not the primary objective
Native deployment	★	Use Native Image instead

23. Final Decision Table

Real-Time Scenario	Recommendation
Serverless Java	☐ Strongly consider AOT
Kubernetes autoscaling	☐ Strongly consider AOT
Frequently restarted microservice	☐ Consider AOT
Containerized Java service	☐ Consider AOT
Java CLI tool	☐ Benchmark and consider
Short-lived batch process	☐ Benchmark and consider
CI/CD Java tool	☐ Benchmark and consider
Long-running backend service	☐ Benchmark first
Application changes every build	☐ Evaluate cache-generation cost
Multiple CPU architectures	☐ Build separate caches
Tiny application with negligible startup	☐ Probably unnecessary
Need a native executable	☐ Consider GraalVM Native Image
Need maximum steady-state throughput only	☐ Focus on JIT/application optimization

Bottom line

JEP 514 is primarily about making Java AOT optimization practical and easier to operate. It doesn't introduce a new Java language feature or API; it makes the JVM's AOT cache workflow considerably easier.



↓
Reduced startup/warm-up

The most important practical rule is:

Use AOT Command-Line Ergonomics when startup and warm-up time materially affect your application—especially serverless, containers, autoscaling services, CLI tools, and frequently restarted workloads. Don't add AOT complexity to a long-running application unless benchmarks demonstrate a meaningful benefit.

And for production, **-XX:AOTMode=auto is generally the safer operational choice**, because the JVM can fall back to normal execution if the cache is unavailable or incompatible. ([Oracle Docs](#))

Java 25 — Ahead-of-Time Method Profiling

Aspect	Details
Feature	Ahead-of-Time Method Profiling
UniqueClassName	AOTMethodProfiling
JEP	JEP 515
Java Version	25
Year	2025
Status	Final
Project	Project Leyden
Primary goal	Improve Java application warm-up by making method-execution profiles from a previous run available when the JVM starts
Main technology	HotSpot JIT + AOT Cache
Primary benefit	JIT can optimize based on previously collected profiles much earlier
Main target	Startup and time-to-peak-performance
Application code changes	Normally none

Runtime model	Still the regular HotSpot JVM; this is not native-image compilation

JEP 515 enhances the Java 25 AOT cache so that **method execution profiles collected during a training run can be stored and reused in subsequent production runs**. This lets the JIT compiler make optimization decisions immediately rather than waiting for the new JVM process to collect enough profile information. ([OpenJDK Mail](#))

1. What Is Ahead-of-Time Method Profiling?

Normally, HotSpot learns how an application behaves **while it is running**.

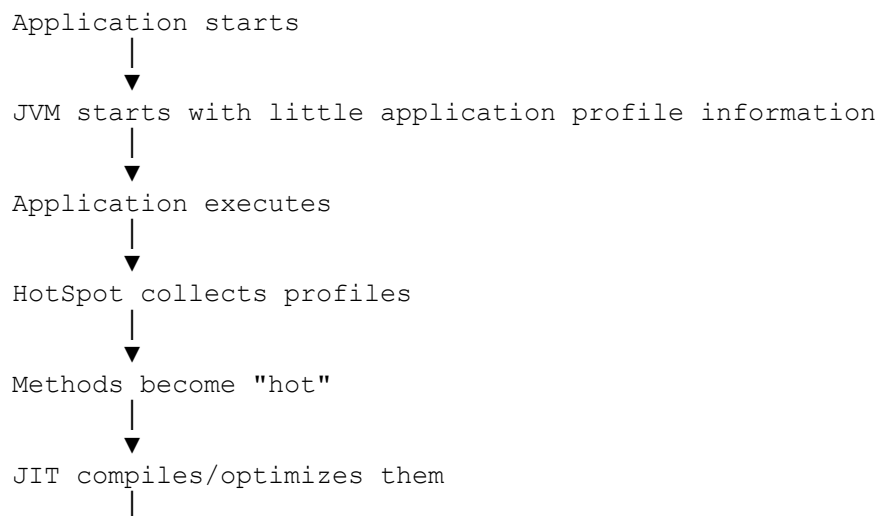
For example:

```
if (customer.isPremium()) {  
    processPremium(customer);  
} else {  
    processStandard(customer);  
}
```

During execution, HotSpot observes things such as:

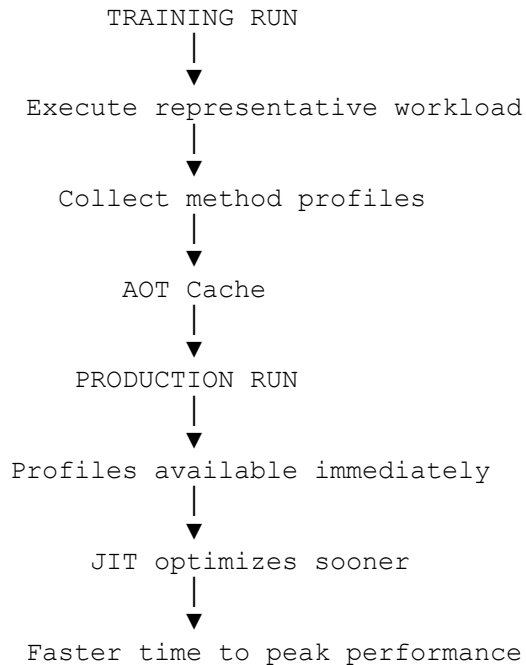
- Which methods are called frequently?
- Which branches are common?
- Which methods become hot?
- Which call sites are stable?
- Which code paths are actually exercised?

The normal process is approximately:



▼
Application reaches peak performance

With JEP 515:

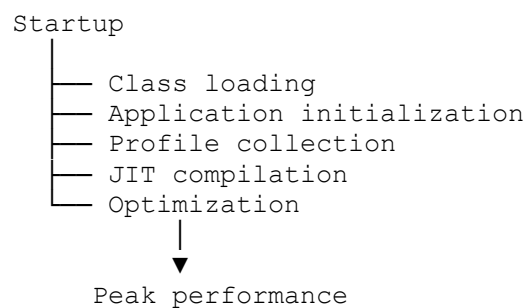


This is the central idea of JEP 515. ([OpenDK Mail](#))

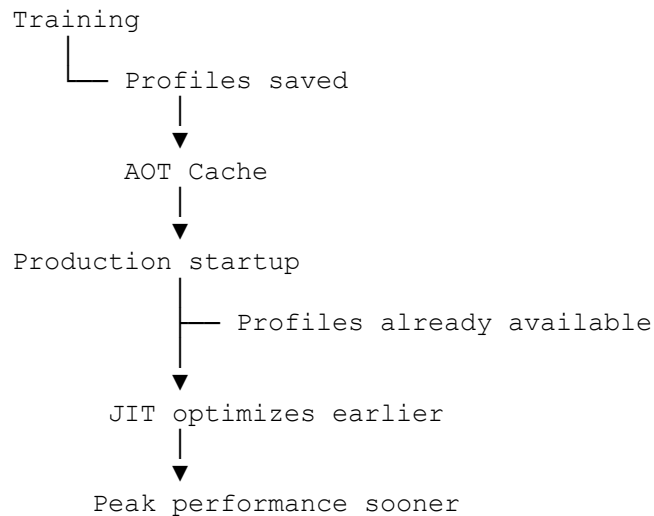
2. Why Is This Important?

Consider a service that needs several seconds before its frequently executed methods become fully optimized.

Without AOT Method Profiling



With AOT Method Profiling



Project Leyden's overall goal is to improve **startup time, time to peak performance, and footprint**. JEP 515 specifically addresses the warm-up portion of that problem. ([OpenJDK](#))

3. Advantages

Advantage	Explanation	Real-world benefit
Faster warm-up	Previous execution profiles are available immediately	Application reaches optimized performance sooner
Earlier JIT optimization	JIT doesn't have to start profiling from zero	Less optimization delay
Better short-lived application performance	Useful when an application terminates before normal JIT warm-up completes	More useful work per execution
Improves frequently restarted services	Each JVM restart can reuse profile information	Less repeated warm-up
Excellent for autoscaling	Newly created JVM instances can benefit from existing profiles	Faster scale-out
Useful for serverless workloads	Short execution windows make warm-up especially important	Better response time
No application-level profiling code required	JVM manages the profile mechanism	Minimal application changes

Works with normal HotSpot JIT	It complements rather than replaces JIT compilation	Retains JVM runtime adaptability
Profile data comes from real execution	Profiles can represent actual workload behavior	Potentially better than purely static guesses
Works with AOT cache infrastructure	Uses the AOT cache introduced in JDK 24	Integrates with Project Leyden
Can complement AOT class/linking optimizations	Class loading/linking and profiling optimizations can work together	Stronger startup/warm-up strategy
Useful for predictable workloads	Stable workload behavior makes profiles more representative	Better optimization quality

The core advantage is that HotSpot can start with profile information from a previous run instead of waiting for a fresh profile to accumulate. ([OpenJDK Mail](#))

4. Disadvantages

Disadvantage	Explanation	Impact
Requires a training run	Profiles must first be collected	Additional build/deployment step
Training workload matters	Poor training data may not represent production	Optimization may be less effective
Profile staleness	Application behavior can change over time	Old profiles may become less useful
Application-specific cache	AOT cache is tied to a particular application	More deployment management
JDK-specific	Cache compatibility depends on the JDK release	Rebuild after JDK changes
Platform-specific	AOT cache is tied to OS/CPU architecture	Separate caches may be needed
Extra build artifact	Profiles live in the AOT cache	Additional storage/deployment handling

Not necessarily useful for long-running applications	Long-running applications eventually collect profiles naturally	Benefit may be small
Doesn't guarantee higher peak performance	It mainly reduces the time needed to reach optimized performance	Benchmark actual workloads
Wrong profiles can hurt optimization quality	Production behavior different from training can reduce usefulness	Training needs representative workloads
Operational complexity	Cache generation becomes part of release engineering	More CI/CD considerations
Not native compilation	The JVM still performs JIT compilation	Doesn't replace Native Image

The AOT cache is specific to the application, JDK release, OS, and CPU architecture, so changes in those dimensions require cache regeneration. ([Oracle Docs](#))

5. When to Use

Situation	Use?	Why
Serverless Java application	☐ Excellent	Execution may finish before normal warm-up
Kubernetes autoscaling	☐ Excellent	New JVMs can reach optimized performance sooner
Frequently restarted microservice	☐ Excellent	Avoids repeated cold profiling
Containerized application	☐ Very good	Predictable deployment environment
CLI application	☐ Good	Short execution makes warm-up significant
CI/CD Java tool	☐ Good	Many short-lived JVM executions
Batch jobs	☐ Good	Useful when jobs don't run long enough to fully warm up
Interactive desktop	☐ Potentially useful	Better early responsiveness

application		
Stable production service	☐ Good	Stable workload produces useful profiles
Large long-running service	☐ Benchmark first	Natural JIT warm-up may already be sufficient
Highly unpredictable workload	☐ Evaluate carefully	Training profiles may not represent production
Frequently changing application	☐ Evaluate carefully	Cache regeneration may outweigh benefits

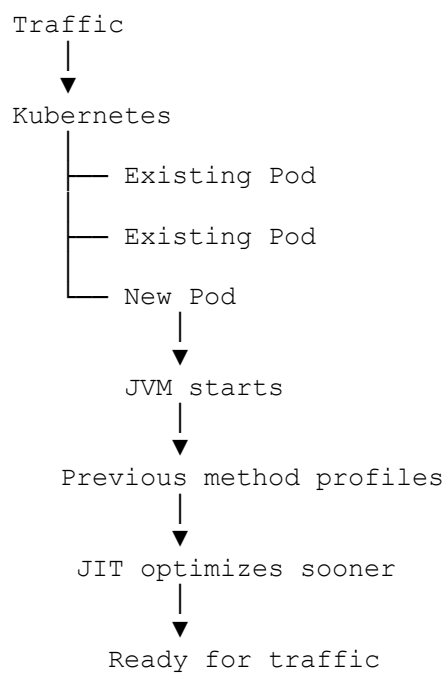
6. When Not to Use

Scenario	Don't use?	Why	Better approach
Application runs for weeks	☐ Usually	Natural JIT profiling has plenty of time	Standard HotSpot JIT
Tiny program with negligible warm-up	☐	Little measurable benefit	Normal JVM
Application changes every build	☐	Frequent cache regeneration	Normal JVM unless benchmark proves value
Highly unpredictable workload	☐	Training profile may become stale	Normal JIT profiling
Development environment	☐ Usually	Code changes frequently	Standard JVM
Multiple heterogeneous architectures	☐	Separate AOT caches may be needed	Build per target
Need native executable	☐	AOT profiling isn't native compilation	GraalVM Native Image
Only peak throughput	☐	Warm-up isn't the primary concern	Focus on JIT/application tuning

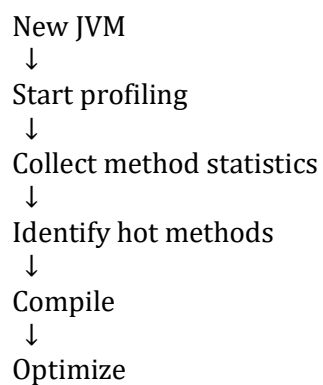
matters			
No representative training workload	☐	Profile quality becomes questionable	Normal JVM or improve training

7. Real-Time Use Case — Kubernetes Autoscaling

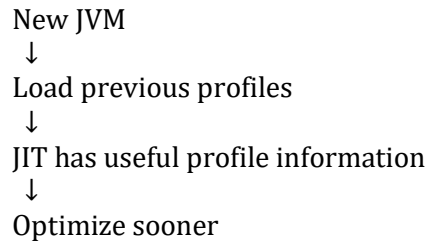
Imagine a Java service:



Without AOT method profiling:



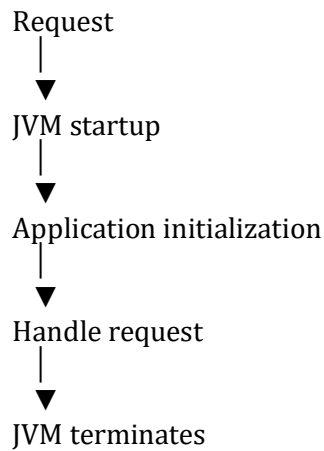
With AOT method profiling:



This is particularly valuable during **rapid scale-out**, where many new JVM instances start simultaneously.

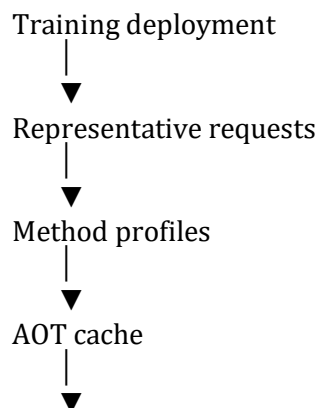
8. Real-Time Use Case — Serverless Java

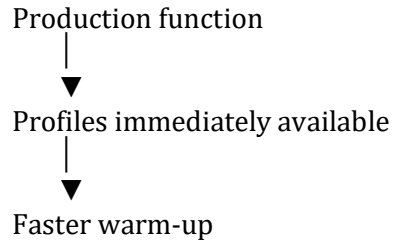
Suppose a serverless Java function executes for only a short period.



The normal JIT may not have enough time to fully optimize the application.

With AOT method profiling:





This is one of the strongest use cases.

9. Real-Time Use Case — Frequently Restarted Microservice

Consider a service restarted during:

- rolling deployments
- Kubernetes rescheduling
- autoscaling
- crash recovery
- infrastructure maintenance

Without profile reuse:

Restart #1 → Profile from zero
Restart #2 → Profile from zero
Restart #3 → Profile from zero
Restart #4 → Profile from zero

With AOT method profiling:

Training
↓
Profile saved
↓
Restart #1 → reuse profile
Restart #2 → reuse profile
Restart #3 → reuse profile
Restart #4 → reuse profile

This reduces the repeated warm-up cost.

10. Real-Time Use Case — REST API

Suppose a REST service frequently executes:

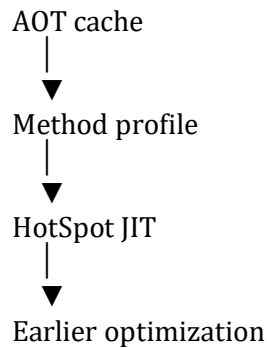
```
void processOrder(Order order) {
    validate(order);
    calculatePrice(order);
    applyDiscount(order);
    save(order);
}
```

During training, the JVM discovers that:

```
processOrder()
├── validate()      frequently called
├── calculatePrice() frequently called
├── applyDiscount() frequently called
└── save()         frequently called
```

The profile information can be retained in the AOT cache.

On a new production JVM:



The important point is that **JEP 515 doesn't simply precompile the entire application into native code**. It gives HotSpot profile information earlier so that its JIT compiler can make better optimization decisions sooner. ([OpenJDK Mail](#))

11. Real-Time Use Case — Java CLI Tool

Consider:

```
java -jar analytics-tool.jar report.csv
```

Suppose the tool performs:

```
Startup
↓
Parse CSV
```

↓
Transform data
↓
Aggregate
↓
Generate report
↓
Exit

If the entire operation takes only a few seconds, normal JVM profiling may spend a meaningful portion of that time collecting information before reaching peak JIT optimization.

AOT method profiling can shift useful profiling information from:

production execution

to:

training execution

making later executions reach optimized behavior sooner.

12. Real-Time Use Case — CI/CD Tool

A Java-based code-analysis tool might run hundreds or thousands of times:

Git commit
↓
CI job
↓
Start Java analyzer
↓
Analyze
↓
Exit

If each execution is short-lived:

Without profile reuse:

Startup → profiling → optimization → work → exit

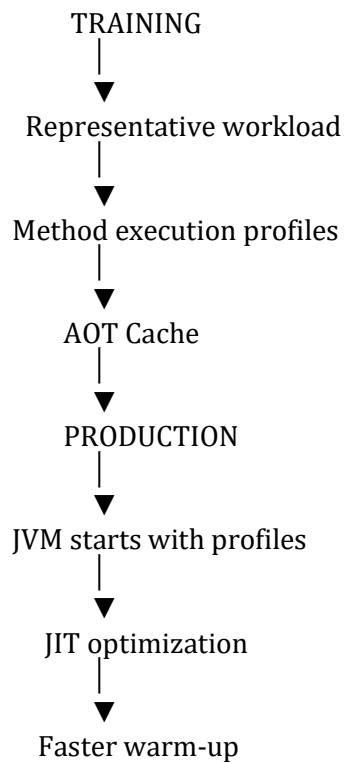
With profile reuse:

Startup → previous profile → earlier optimization → work → exit

For high-volume CI environments, even small improvements in startup/warm-up can accumulate.

13. Training vs Production

A representative workflow is:



The training workload should resemble real production behavior.

For example:

Training workload	Production workload	Result
Real API requests	Same API patterns	☐ Excellent
Real business transactions	Similar transactions	☐ Excellent
Random synthetic workload	Very different production traffic	☐ Poor
Only startup execution	Complex production behavior	☐ Limited
Representative benchmark	Similar production workload	☐ Good

14. AOT Method Profiling vs Normal JIT Profiling

Aspect	Normal JIT Profiling	AOT Method Profiling
Profile collected during execution	✓	✓ During training
Profile available at JVM startup	✗	✓
Requires training run	✗	✓
Warm-up time	Higher	Potentially lower
Runtime adaptability	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Deployment complexity	★ ★ ★ ★ ★	★ ★ ★
Best for long-running services	★ ★ ★ ★ ★	★ ★ ★
Best for short-lived workloads	★ ★ ★	★ ★ ★ ★ ★
Profile reuse	✗	✓
JVM still uses JIT	✓	✓
Native executable	✗	✗

15. AOT Method Profiling vs AOT Class Loading & Linking

These are complementary Java features.

Aspect	JEP 483 — AOT Class Loading & Linking	JEP 515 — AOT Method Profiling
Main concern	Class startup work	Method optimization warm-up
Prepares classes	✓	Not primary purpose
Prepares linking	✓	Not primary purpose

Stores method profiles	✗	✓
Helps startup	★★★★★	★★★
Helps warm-up	★★★★	★★★★★
Helps JIT optimization	Indirectly	Directly
Main benefit	Less startup work	Earlier optimized execution

A useful mental model is:

JEP 483

"Start the JVM with more startup work already done."

JEP 515

"Start the JVM with useful method behavior already known."

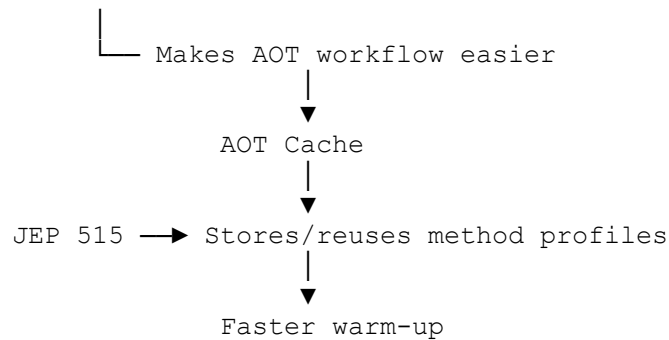
16. AOT Method Profiling vs AOT Command-Line Ergonomics

The previous feature you asked about, **JEP 514**, and JEP 515 complement each other.

Aspect	JEP 514	JEP 515
Feature	AOT Command-Line Ergonomics	AOT Method Profiling
Main purpose	Simplify AOT workflow	Reuse method profiles
Focus	Developer/deployment experience	JVM warm-up
AOT cache	Uses it	Enhances it
Profile storage	Not primary purpose	Yes
Faster warm-up	Indirect	Primary goal
Easier cache creation	Primary goal	Uses same infrastructure
Can be used together	✓	✓

So:

JEP 514



Project Leyden lists both JEP 514 and JEP 515 among the features delivered for JDK 25. ([OpenJDK](#))

17. Alternate Features / Technologies

There isn't one exact replacement. The best alternative depends on what you are trying to optimize.

Requirement	Alternative	When preferable
Normal JVM optimization	HotSpot JIT profiling	Long-running applications
Faster startup	CDS	When class sharing is sufficient
Faster class loading/linking	JEP 483 AOT Class Loading & Linking	Startup is the primary bottleneck
Easier AOT deployment	JEP 514	AOT workflow complexity is the problem
Native executable	GraalVM Native Image	JVM-free deployment is required
Native Image optimization	Profile-Guided Optimization (PGO)	Native executable + runtime profile information
Application-level optimization	JFR / async-profiler	Need to identify actual runtime bottlenecks
Long-running workload	Normal JIT + profiling	Application naturally reaches peak performance

GraalVM's PGO is conceptually related because it also uses runtime behavior to improve compiled performance, but it belongs to a different compilation/deployment model. ([Oracle Docs](#))

18. AOT Method Profiling vs GraalVM Native Image PGO

Aspect	JEP 515	GraalVM Native Image PGO
Target	HotSpot JVM	Native executable
JVM required	✓	✗ at runtime
Uses runtime profiles	✓	✓
Profile from training	✓	✓
JIT remains active	✓	✗
Native executable	✗	✓
Java compatibility	High	Requires Native Image compatibility
Best for	JVM applications	Native deployments
Startup optimization	★ ★ ★ ★ ★	★ ★ ★ ★ ★
Warm-up optimization	★ ★ ★ ★ ★	Different model
Deployment complexity	Moderate	Higher

19. Is It a Performance Feature?

Yes — but specifically a startup/warm-up performance feature.

It should not be interpreted as:

"Every Java application will run faster."

Instead:

The application can reach its optimized performance state sooner.

Performance area	Expected value
Startup	★ ★ ★
Time to first useful execution	★ ★ ★ ★

Warm-up	★ ★ ★ ★ ★
Time to peak performance	★ ★ ★ ★ ★
Peak steady-state throughput	★ ★ – ★ ★ ★
Long-running workload	★ ★ – ★ ★ ★
Short-lived workload	★ ★ ★ ★ ★
Autoscaling	★ ★ ★ ★ ★
Serverless	★ ★ ★ ★ ★

The distinction is important: JEP 515's goal is specifically to reduce the profiling delay before the JIT can generate optimized native code. ([OpenJDK Mail](#))

20. AOT Cache Compatibility

Java 25's AOT cache is tied to:

Application
 +
 JDK release
 +
 Operating system
 +
 CPU architecture

If one changes:

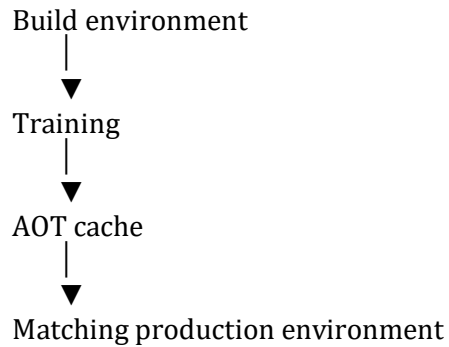
JDK 25
 ↓
 JDK 26

or:

x86-64
 ↓
 ARM64

or the application changes substantially, the cache generally needs to be recreated. ([Oracle Docs](#))

Therefore:



is the safest model.

21. Production Deployment Recommendation

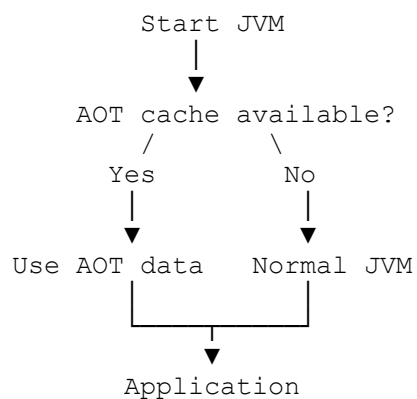
For production, Java 25's AOT infrastructure provides:

`-XX:AOTMode=auto`

as the default AOT mode.

In auto mode, the JVM can use the AOT cache if it is usable and otherwise continue without it. Oracle specifically recommends this mode for production because an incompatible cache shouldn't normally prevent the application from running. ([Oracle Docs](#))

Conceptually:



22. Overall Assessment

Category	Rating	Assessment
Warm-up improvement	★★★★★	Primary strength

Time-to-peak performance	★ ★ ★ ★ ★	Primary purpose
Serverless	★ ★ ★ ★ ★	Excellent candidate
Autoscaling	★ ★ ★ ★ ★	Excellent candidate
Frequently restarted services	★ ★ ★ ★ ★	Very useful
CLI applications	★ ★ ★ ★	Often beneficial
CI/CD tools	★ ★ ★ ★	Potentially valuable
Stable microservices	★ ★ ★ ★	Good candidate
Long-running services	★ ★ ★	Benchmark first
Unpredictable workloads	★ ★	Profile quality may suffer
Operational simplicity	★ ★ ★	Requires AOT-cache workflow
Portability	★ ★	Cache is environment-specific
Peak throughput	★ ★ ★	Not the primary target

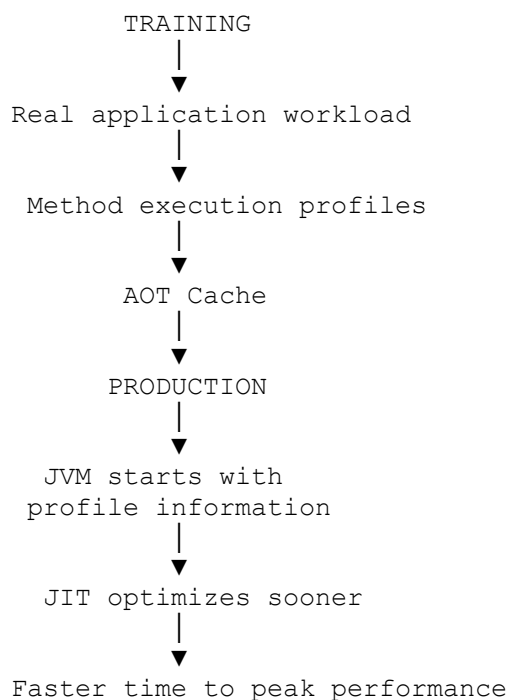
23. Final Decision Table

Real-Time Scenario	Recommendation
Serverless Java	☐ Strongly consider
Kubernetes autoscaling	☐ Strongly consider
Frequently restarted microservice	☐ Strongly consider
Containerized service	☐ Consider
Short-lived Java application	☐ Consider
CLI application	☐ Benchmark and consider
CI/CD Java tool	☐ Benchmark and consider

Stable REST API	☐ Benchmark and consider
Long-running backend	☐ Benchmark first
Highly unpredictable workload	☐ Evaluate carefully
Constantly changing application	☐ Cache maintenance may outweigh benefit
Tiny application	☐ Probably unnecessary
Need native executable	☐ Use Native Image instead
Only peak throughput matters	☐ Focus on normal JIT/application optimization

Bottom line

JEP 515 — Ahead-of-Time Method Profiling — is primarily a warm-up optimization. It lets HotSpot start with method-execution profiles obtained from an earlier representative run, so the JIT can make useful optimization decisions sooner instead of spending the beginning of every production JVM run rediscovering the application's behavior.



Best rule:

Use AOT Method Profiling when your Java application is short-lived, frequently restarted, rapidly autoscaled, or sensitive to warm-up time. For long-running applications that

naturally reach peak performance quickly enough, benchmark first before accepting the additional AOT-cache build and deployment complexity.

Java 25 — JFR Cooperative Sampling

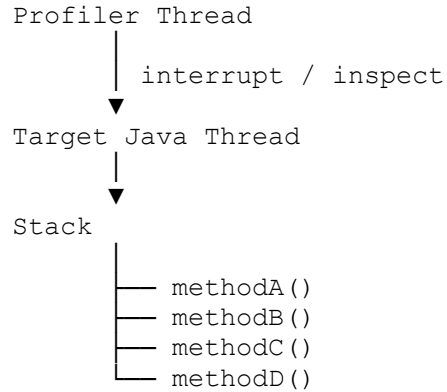
Aspect	Details
Feature	JFR Cooperative Sampling
UniqueClassName	JFRCooperativeSampling
JEP	JEP 518
Java Version	25
Year	2025
Status	Final
Main technology	Java Flight Recorder (JFR)
Primary goal	Make JFR execution sampling more efficient and robust by using cooperative stack walking
Main target	CPU profiling and execution-stack sampling
Main benefit	Reduce the overhead and complexity associated with asynchronously walking Java thread stacks
Application code changes	Normally none
Performance impact	Primarily improves profiling overhead/implementation; it does not make the application itself inherently faster

JEP 518 changes JFR's Java-thread stack sampling from an asynchronous stack-walking approach to a **cooperative sampling mechanism**. Instead of the sampler forcibly walking a thread's stack at arbitrary points, the target thread reaches safe sampling points and cooperates with JFR. This is especially useful for reducing profiling overhead and improving sampling robustness.

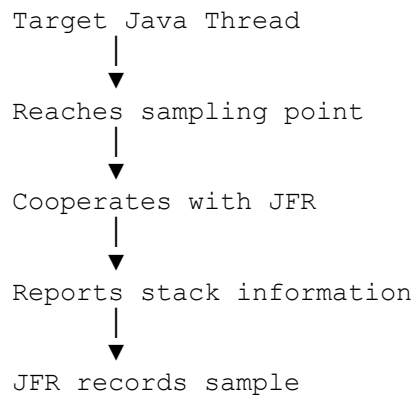
1. What Is JFR Cooperative Sampling?

Java Flight Recorder (JFR) collects information about a running JVM with relatively low overhead.

Traditional sampling can conceptually be viewed as:



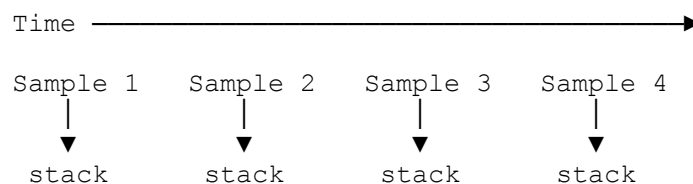
JFR Cooperative Sampling changes the model toward:



So instead of the profiler doing as much work as possible **to** the application thread, the application thread reaches appropriate points where it can safely participate in the sampling process.

2. Why Is This Useful?

CPU profiling typically works by taking many samples:



From those samples JFR can determine:

methodA() → 5%

methodB() → 20%
methodC() → 50%
methodD() → 25%

This provides a statistical picture of where CPU time is being spent.

JEP 518 improves how JFR obtains these samples by using **cooperative stack sampling**, reducing reliance on asynchronous stack walking.

3. Traditional Sampling vs Cooperative Sampling

Aspect	Traditional JFR Sampling	JFR Cooperative Sampling
Sampling model	Asynchronous	Cooperative
Target thread cooperation	Limited	Yes
Stack walking	Asynchronous stack walking	Cooperative stack walking
Profiling overhead	Higher in some situations	Reduced
Robustness	More complexity around arbitrary execution points	Improved
Application source changes	None	None
JFR integration	Existing	Native JFR improvement
Primary purpose	Obtain execution samples	Obtain them more efficiently/safely
Makes application inherently faster	✗	✗
Useful for production profiling	✓	✓

4. Advantages

Advantage	Explanation	Real-world benefit
Lower sampling	Cooperative sampling	Less profiler impact

overhead	avoids some costs of asynchronous stack walking	
Improved profiling efficiency	JVM threads participate in sampling at appropriate points	More efficient stack collection
Better robustness	Avoids some difficult cases associated with asynchronous stack walking	More reliable profiling
Good for continuous JFR	Lower overhead is valuable when recording for long periods	Production monitoring becomes more practical
No application-code changes	The JVM/JFR handles the mechanism	Easy adoption
Works with JFR's existing model	Developers continue using JFR tooling	Minimal operational disruption
Useful for CPU profiling	Execution samples show where CPU-consuming code runs	Easier CPU bottleneck identification
Useful for heavily threaded applications	Many threads can otherwise make stack sampling expensive	Better scalability of profiling
Works well with modern Java concurrency	Valuable for applications using large numbers of threads/virtual threads	Better observability
Can reduce profiling perturbation	Less invasive sampling can give a more representative workload picture	Better performance diagnosis
Production-friendly	JFR is designed for low-overhead production recording	Easier troubleshooting in live systems
Complements JFR CPU-time profiling	Sampling and CPU-time measurements can answer different questions	More complete performance analysis

5. Disadvantages

Disadvantage	Explanation	Impact
Still sampling-based	It provides statistical information rather than a complete execution trace	Very short operations may be missed
Not exact CPU accounting by itself	A sample indicates where execution was observed, not exact CPU duration	Use CPU-time events when exact CPU time matters
Sampling frequency matters	Too few samples reduce statistical accuracy	Important methods may be missed
Not an application optimization	Cooperative sampling doesn't improve application logic automatically	Developer still has to optimize code
JFR knowledge required	Developers need to understand recordings/events/stacks	Learning curve
Can still have overhead	Profiling is not completely free	High-frequency recording should be benchmarked
Profile interpretation required	A large method sample count doesn't automatically mean it is the root cause	Requires performance analysis
Sampling can miss rare events	Infrequently executed code may not appear in samples	Use event-based diagnostics where appropriate
Not a replacement for tracing	Sampling and tracing answer different questions	Use appropriate observability tools
Not a memory profiler by itself	CPU stack sampling doesn't identify every memory problem	Use JFR allocation/GC events and other tools

6. When to Use

Situation	Use?	Why
-----------	------	-----

CPU performance investigation	🔍 Excellent	Shows where execution time is concentrated
Production CPU profiling	🔍 Excellent	JFR is designed for low-overhead recording
High-throughput REST service	🔍 Excellent	Identifies hot application paths
Microservices	🔍 Excellent	Useful for diagnosing CPU bottlenecks
Virtual-thread applications	🔍 Very good	Large concurrency makes efficient sampling valuable
Long-running JVM	🔍 Very good	Continuous/periodic recordings can reveal trends
Performance regression analysis	🔍 Excellent	Compare stack profiles between versions
JIT investigation	🔍 Good	Helps locate frequently executed application code
Algorithm optimization	🔍 Excellent	Finds CPU-heavy methods
Production incident investigation	🔍 Excellent	Capture evidence without invasive instrumentation
Memory leak investigation	🔍 Not primary tool	Use JFR memory/GC events too
Exact CPU accounting	🔍 Use with CPU-time profiling	Sampling is statistical

7. When Not to Use

Scenario	Don't use as primary tool?	Better approach
Exact CPU-time accounting	🔍	JFR CPU-Time Profiling
Memory leak investigation	🔍	JFR allocation/GC analysis, heap dump

Deadlock investigation	?	JFR monitor/thread events, thread dumps
Distributed request tracing	?	OpenTelemetry/distributed tracing
Single rare event	?	Event-based JFR recording
Application correctness testing	?	Tests/logging/debugging
Extremely tiny workload	?	Direct benchmark/profiler
Need complete execution history	?	Instrumentation/tracing
Need line-by-line deterministic profiling	?	Instrumentation-based profiler
GPU profiling	?	GPU-specific profiling tools

8. Real-Time Use Case — CPU-Hungry REST API

Suppose your service exposes:

```

POST /orders
  ↓
processOrder()
  ├── validate()
  ├── calculatePrice()
  ├── applyDiscount()
  ├── serialize()
  └── save()

```

The service is experiencing high CPU usage.

Start a JFR recording and inspect execution samples.

You might discover:

calculatePrice()		45%
JSON serialization		25%
validate()		15%
other		15%

You now know where to investigate first.

Possible optimization:

Before:
`calculatePrice()` → expensive loop

After:
`calculatePrice()` → optimized algorithm

The value of cooperative sampling is that the profiling process itself is designed to be less intrusive.

9. Real-Time Use Case — Microservice CPU Regression

Suppose:

Version 1.0
CPU = 40%

Version 1.1
CPU = 70%

You can record both versions:

Version 1.0
↓
▼
JFR Recording
↓
▼
CPU Stack Profile

Version 1.1
↓
▼
JFR Recording
↓
▼
CPU Stack Profile

Comparison:

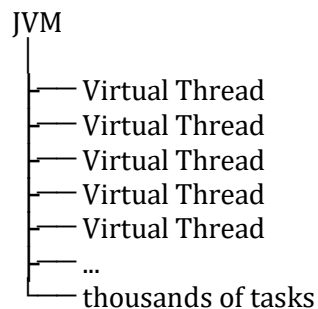
Method	v1.0 samples	v1.1 samples	Observation
<code>processOrder()</code>	15%	16%	Similar

calculatePrice()	20%	45%	⌘ Regression
serialize()	10%	12%	Small change
Other	55%	27%	Reduced

This immediately directs the investigation toward `calculatePrice()`.

10. Real-Time Use Case — Virtual Threads

Consider a Java 25 application:



Suppose CPU usage unexpectedly increases.

JFR sampling can help determine which execution paths consume CPU.

For example:

HTTP processing	20%
JSON serialization	35%
Business calculation	30%
Logging	5%
Other	10%

You can then focus optimization on the actual CPU-heavy paths instead of guessing.

11. Real-Time Use Case — Inefficient Algorithm

Suppose:

```
List<String> users = ...;
```

```
for (String user : users) {
    if (users.contains(user)) {
        ...
    }
}
```

For a large collection, this could cause significant CPU consumption.

JFR sampling may reveal:

```
UserService.process()
├── List.contains()
│   └── large percentage of samples
```

You might replace the data structure/algorithm:

```
Set<String> users = ...;
```

The profiler doesn't make the optimization for you—it identifies where to investigate.

12. Real-Time Use Case — JSON Serialization

A REST application might spend a surprising amount of CPU in serialization.

JFR samples:

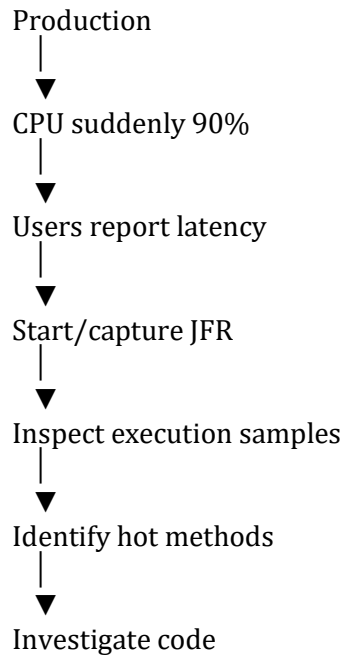
```
Application
├── API response
│   └── JSON serialization
│       ├── reflection
│       ├── object traversal
│       └── string processing
```

If JFR shows serialization consuming a significant fraction of samples, you can investigate:

- excessive object creation
- unnecessary fields
- repeated serialization
- inefficient serialization configuration
- large response payloads

13. Real-Time Use Case — Production Incident

Imagine:



This is one of the major strengths of JFR:

You can investigate JVM behavior using production-oriented recordings rather than immediately reproducing everything in a development environment.

14. Real-Time Use Case — JIT Investigation

Suppose a method unexpectedly consumes substantial CPU:

```
int calculateTotal(List<Order> orders) {  
    ...  
}
```

JFR can help establish:

```
calculateTotal()  
├── called frequently  
├── significant CPU samples  
└── candidate for optimization
```

You can then investigate:

- method size
- allocations
- call frequency
- polymorphism
- synchronization
- algorithmic complexity

This is much more useful than blindly changing JVM flags.

15. Cooperative Sampling vs JFR CPU-Time Profiling

These features are related but **not identical**.

Aspect	JFR Cooperative Sampling	JFR CPU-Time Profiling
JEP	518	509
Java version	25	25
Main purpose	Efficient stack sampling	CPU-time measurement
Sampling	✓	Can be used for CPU profiling
CPU time	Indirect/statistical	Direct measurement
Stack information	★★★★★	★★★★
Exact CPU accounting	✗	Better suited
Profiling overhead	Improved sampling mechanism	CPU-time measurement
Find hot methods	★★★★★	★★★★★
Distinguish CPU vs waiting	Limited by sampling	Strong
Best use	Execution-stack profiling	CPU-time analysis

A useful mental model:

JFR Cooperative Sampling



"Where is the application executing?"

JFR CPU-Time Profiling



"How much CPU time is being consumed?"

Using both can provide a stronger picture.

16. Cooperative Sampling vs JFR Event Recording

Aspect	Sampling	Event Recording
Captures stacks periodically	✓	Sometimes
Captures every event	✗	✓
Low overhead	★★★★★	★★★★
Exact occurrence history	✗	✓
Find CPU hotspots	★★★★★	★★★
Find GC pauses	★★	★★★★★
Find allocation events	★★	★★★★★
Find rare events	★★	★★★★★
Long-term profiling	★★★★★	★★★★

17. Cooperative Sampling vs Async-Profiler

async-profiler is a popular external profiler.

Aspect	JFR Cooperative Sampling	async-profiler
Built into JDK	✓	✗
JFR integration	Native	Can produce JFR-compatible output
CPU profiling	✓	✓

Low overhead	★★★★★	★★★★★
Advanced native profiling	Good	Excellent
Flame graphs	Via JMC/other tools	Excellent
Production use	★★★★★	★★★★★
Setup complexity	Low	Moderate
Best for	Integrated JVM observability	Deep profiling

For many production JVM investigations, JFR is a very convenient first tool because it is already part of the JDK.

18. Alternate Features / Technologies

There is no single exact replacement because different tools answer different performance questions.

Requirement	Alternative	When preferable
CPU stack profiling	async-profiler	Need advanced CPU/native profiling
JVM event analysis	JFR	Need integrated JVM diagnostics
Exact CPU-time analysis	JFR CPU-Time Profiling	Need CPU vs waiting distinction
Interactive JVM monitoring	JMX	Need live management metrics
Heap analysis	Heap Dump	Need object-retention investigation
Allocation profiling	JFR allocation events	Need allocation hotspots
Distributed tracing	OpenTelemetry	Need cross-service request flow
Simple system CPU metrics	OS tools	Need whole-machine CPU information
Deterministic profiling	Instrumentation profiler	Need precise method invocation measurements

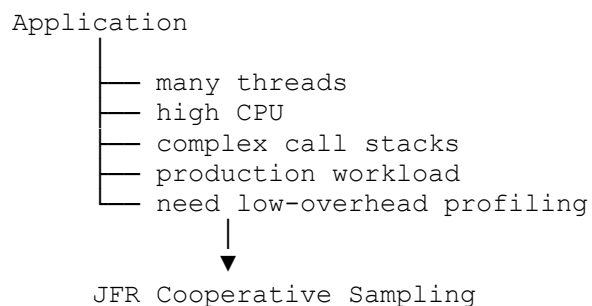
Native-code profiling	perf / async-profiler	Need OS/native stack detail

19. JFR Cooperative Sampling vs Traditional Logging

Requirement	Logging	JFR Cooperative Sampling
Application events	★★★★★	★★★
CPU hotspots	★	★★★★★
Method stacks	★★	★★★★★
Production overhead	Variable	Low
Historical performance evidence	★★★	★★★★★
Automatic JVM information	★★	★★★★★
Debugging business logic	★★★★★	★★★
Performance investigation	★★	★★★★★

20. When It Is Especially Valuable

The feature becomes particularly valuable when:



Examples:

Application	Value
-------------	-------

High-throughput REST API	★★★★★
Microservices	★★★★★
Virtual-thread service	★★★★★
Messaging service	★★★★★
Financial transaction processing	★★★★★
Large JVM application	★★★★★
Batch processor	★★★★
CLI utility	★★★
Tiny Java program	★
Simple CRUD application with no performance issue	★★

21. Is It a Performance Optimization Feature?

This distinction is important.

JFR Cooperative Sampling is primarily an observability/profiling improvement.

It doesn't optimize:

your algorithm
 your database query
 your JSON processing
 your network calls
 your GC configuration

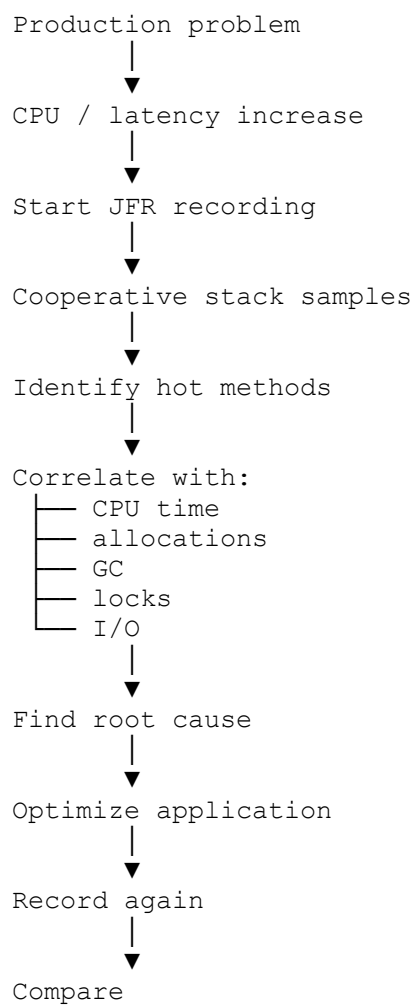
Instead, it improves your ability to **measure and understand** those things with lower profiling overhead.

Area	Direct benefit
Application throughput	✗
Application latency	✗
CPU usage	✗

GC performance	✗
JIT optimization	✗
Profiling overhead	✓
Stack sampling efficiency	✓
Production observability	✓
Performance diagnosis	★★★★★

22. Recommended Production Workflow

A good workflow is:



This is much better than changing JVM flags based solely on assumptions.

23. Overall Assessment

Category	Rating	Assessment
CPU profiling	★★★★★	Excellent
Stack profiling	★★★★★	Excellent
Production diagnostics	★★★★★	Excellent
Low-overhead profiling	★★★★★	Major strength
Microservices	★★★★★	Excellent
Virtual threads	★★★★★	Very useful
CPU regression analysis	★★★★★	Excellent
Algorithm optimization	★★★★★	Excellent
Exact CPU accounting	★★★★	Pair with CPU-time profiling
Memory analysis	★★★	Not primary purpose
Distributed tracing	★★	Use tracing tools
Runtime application speed	★	Not itself an optimization

24. Final Decision Table

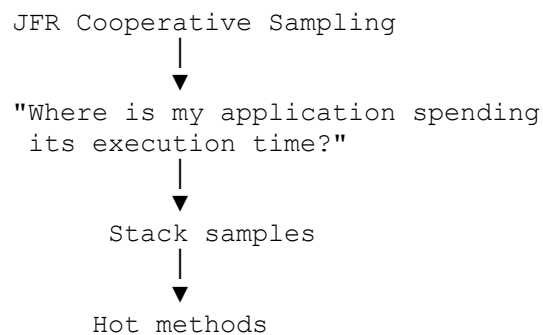
Real-Time Scenario	Recommendation
Production CPU investigation	🔍 Use
High CPU REST API	🔍 Use
Microservice performance analysis	🔍 Use
Virtual-thread application profiling	🔍 Use
CPU regression between releases	🔍 Use

Algorithm optimization	🔍 Use
JIT/hot-method investigation	🔍 Use
Production incident diagnosis	🔍 Use
Long-running JVM profiling	🔍 Use
Exact CPU-time measurement	🔍 Use with JFR CPU-Time Profiling
Memory leak	🔍 Use JFR memory/heap tools instead/as well
Distributed request flow	🔍 Use distributed tracing
Rare one-off event	🔍 Use JFR event recording
Need native-code deep profiling	🔍 Consider async-profiler/perf
Tiny application with no performance problem	🔍 Probably unnecessary

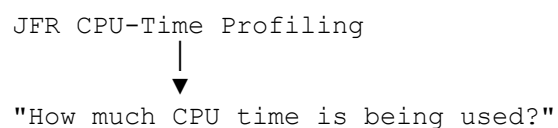
Bottom line

JFR Cooperative Sampling is a Java 25 observability improvement that makes JFR's execution-stack sampling more efficient by having Java threads cooperate with the sampling process.

The most important distinction is:



while:



And:

JFR Events



"What JVM/application events happened?"

So the practical rule is:

Use JFR Cooperative Sampling when you need low-overhead, production-oriented CPU/stack profiling. Combine it with JFR CPU-Time Profiling and other JFR events when you need to distinguish CPU consumption from waiting, allocation, GC, locking, and I/O behavior.

Java 25 — Compact Object Headers

Aspect	Details
Feature	Compact Object Headers
UniqueClassName	CompactObjectHeaders
JEP	JEP 519
Java Version	25
Year	2025
Status	Final / Product Feature
Origin	JEP 450 — Experimental in Java 24
Project	Project Lilliput
Main JVM option	-XX:+UseCompactObjectHeaders
Primary goal	Reduce the memory overhead of every Java object
Traditional header	96 or 128 bits on supported 64-bit configurations
Compact header	64 bits
Main benefit	Lower heap usage and potentially better cache locality/deployment density

Application code changes	None
Default in Java 25?	No — it remains opt-in
Important limitation	Maximum of about 4 million loaded classes when enabled

JEP 519 promotes Compact Object Headers from the experimental feature introduced by JEP 450 in Java 24 to a product feature in Java 25. Importantly, **JEP 519 does not make it the default object-header layout**; applications must opt in with `-XX:+UseCompactObjectHeaders`. ([Oracle](#))

1. What Are Object Headers?

Every Java object in the heap has metadata associated with it.

Conceptually:

Traditional Java object

Object Header Mark / lock / GC metadata Class information
Instance fields
Padding

For many small objects, the header itself can represent a significant percentage of the object's total size.

For example, conceptually:

Small object

Header
Fields



When an application has **millions or billions of small objects**, the cumulative header overhead can become substantial.

2. What Does Compact Object Headers Change?

Compact Object Headers reduce the object-header size to **64 bits (8 bytes)** from 96 or 128 bits on the relevant 64-bit HotSpot configurations. ([Oracle Docs](#))

Conceptually:

Traditional

Object Header 96 / 128 bits

Compact

Object Header 64 bits

The JVM accomplishes this by packing object metadata more efficiently and using auxiliary structures when metadata such as locking or identity-hash information cannot fit directly into the compact header.

The important point for Java developers is:

Your Java object model does not change. The JVM changes the physical representation of objects.

3. How Much Can It Matter?

Suppose an application retains:

100 million objects

and the header saving were approximately:

4 bytes/object

Then the theoretical header saving alone could be:

$100,000,000 \times 4 \text{ bytes}$
 $\approx 381 \text{ MiB}$

This is an **illustrative calculation**, not a guaranteed application-level saving.

Actual memory reduction depends on:

- object sizes,
- alignment/padding,
- number of live objects,
- object graph,
- compressed references,
- garbage collector,
- allocation behavior,
- class usage.

Oracle states that reducing the header size reduces live-data size, potentially improves data locality, and can increase deployment density. ([Oracle Docs](#))

4. Advantages

Advantage	Explanation	Real-world benefit
Lower per-object memory overhead	Each object can have a smaller header	Lower heap consumption
Excellent for many small objects	Header overhead becomes significant when object count is high	Large memory savings possible
Better cache locality	Smaller objects mean more objects can fit into cache/heap regions	Potential performance improvement
Lower live-set size	Same logical object graph can require less memory	Less memory pressure
Potentially lower GC pressure	Smaller heap footprint can reduce memory pressure	Potentially fewer/cheaper collections
Higher deployment density	More workloads can fit into the same machine/container memory	Lower infrastructure cost
Good for cloud environments	Memory is often a major container/serverless constraint	Better resource utilization
No application code changes	JVM handles object representation internally	Easy to test
Works transparently	Java semantics remain unchanged	Existing code generally doesn't need modification

Product feature in Java 25	No longer experimental	Suitable for production evaluation
Useful for object-heavy frameworks	Frameworks often create many objects	Potentially significant savings
Can improve performance indirectly	Smaller objects and less memory pressure may improve locality/GC behavior	Potential throughput/latency gains

Oracle specifically identifies reduced object size, reduced live-data size, higher deployment density, and increased data locality as benefits. ([Oracle Docs](#))

5. Disadvantages

Disadvantage	Explanation	Impact
Not enabled by default	Java 25 requires explicit opt-in	Must deliberately test and deploy it
Class-count limitation	Enabled applications have a limit of about 4 million different loaded classes	Problem for extremely class-heavy systems
Potential workload-dependent results	Memory reduction doesn't necessarily translate into faster execution	Benchmark required
Some metadata moves outside the header	Certain operations may need auxiliary metadata	Potential overhead in specific cases
Object-layout sensitivity	Very low-level JVM feature	Low-level tools may need validation
Not useful for large primitive arrays	Header is tiny relative to a huge array's payload	Little benefit
Not useful for off-heap memory	Direct buffers/native memory aren't Java heap objects in the same way	Little/no direct benefit
No code-level control	Developers can't selectively compact only certain classes	JVM-wide behavior
JVM compatibility	Some low-level JVM	Deployment testing required

considerations	options/configurations may interact with it	
Could complicate troubleshooting	Changing object layout changes low-level runtime behavior	Compare with/without during diagnosis
No guaranteed CPU improvement	Memory reduction doesn't automatically mean faster execution	Avoid assuming performance gains
Requires Java 25+	Earlier Java versions need their corresponding experimental/other implementation	Version dependency

Oracle explicitly documents the approximately **4-million loaded-class limit** when compact headers are enabled. ([Oracle Docs](#))

6. When to Use

Situation	Use?	Reason
Millions of small Java objects	👍 Excellent	Header overhead becomes significant
Large object-heavy microservice	👍 Excellent candidate	Can reduce heap footprint
Spring-based application with many objects	👍 Consider	Framework/object graphs can be object-dense
Large collections/maps	👍 Consider	Many individual objects may be involved
JSON-heavy processing	👍 Consider	Parsing/deserialization can create many short-lived objects
Messaging systems	👍 Consider	Message/domain objects can be numerous
Caches with many Java objects	👍 Consider	Large object count
Graph processing	👍 Excellent candidate	Graphs often contain huge numbers of small nodes/edges

AST/compiler workloads	🔍 Excellent candidate	ASTs contain many small objects
Memory-constrained containers	🔍 Excellent candidate	Every MB of heap matters
High-density Kubernetes deployment	🔍 Consider	Lower memory can allow more pods per node
Serverless Java	🔍 Consider	Memory allocation affects cost/resource limits
Large object count but low memory pressure	🔍 Benchmark	Benefit may be modest
Large primitive arrays	🔍 Usually little benefit	Payload dominates memory

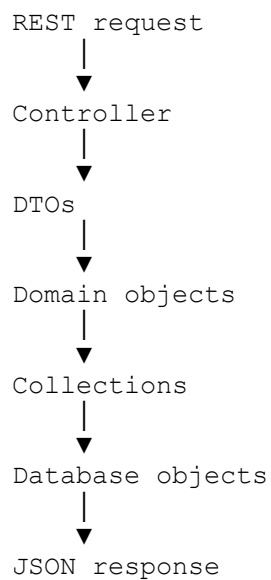
7. When Not to Use

Scenario	Don't use?	Why	Better approach
Application dominated by large arrays	🔍 Usually	Header is tiny relative to payload	Optimize data structures
Mostly off-heap memory	🔍	Compact headers don't reduce off-heap memory	Optimize native/direct memory
Very few Java objects	🔍	Header savings are insignificant	Default configuration
Extremely large number of loaded classes	🔍	4-million-class limit	Standard headers
Application already has ample memory	🔍	Benefit may not justify testing	Benchmark first
Need guaranteed CPU improvement	🔍	Not guaranteed	Profile CPU/JIT/application
Application has no memory problem	🔍	Optimization may be unnecessary	Keep baseline
Production rollout	🔍	Low-level JVM	Benchmark/staged

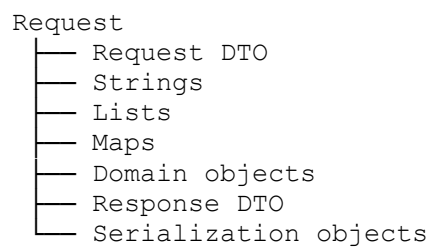
without testing		change should be validated	rollout
JVM troubleshooting	?	Keep a baseline without the option	Compare both configurations

8. Real-Time Use Case — Large Microservice

Imagine a Spring-based service:



A single request can create many objects:



Now multiply that by:

10,000 requests/sec

The application may create a very large number of objects.

Compact Object Headers can reduce the per-object overhead.

Deployment

```
java -XX:+UseCompactObjectHeaders \  
-jar order-service.jar
```

Then compare:

Standard headers
vs
Compact headers

using:

- JFR
 - GC logs
 - heap usage
 - allocation rate
 - latency
 - throughput
-

9. Real-Time Use Case — Large Cache

Suppose a cache stores:

```
Map<Long, Customer> customers;
```

with:

10 million Customer objects

Each customer might contain:

```
Customer  
├── String  
├── Address  
├── Preferences  
├── Account  
└── other objects
```

The object graph becomes enormous:

```
10M Customer  
├── 10M+ Address  
├── 10M+ Preference  
└── ...
```

└ many millions of objects

Compact headers become particularly interesting because the memory cost is driven by **object count**, not simply by the size of individual fields.

10. Real-Time Use Case — Graph Processing

Graph algorithms often represent data as many small objects:

```
Node
├ id
├ neighbors
└ metadata
```

A graph with:

100 million nodes

can have an enormous object count.

Conceptually:

Traditional:

```
Node + Header
Node + Header
Node + Header
...
100M times
```

Compact:

```
Node + smaller Header
Node + smaller Header
Node + smaller Header
...
100M times
```

This is a strong candidate for compact object headers.

However, for extremely large graphs, you should also consider **flattened/off-heap/primitive-oriented data structures**, because changing the object header alone may not be enough.

11. Real-Time Use Case — JSON Deserialization

Consider a large JSON document:

```
{
  "customers": [
    {
      "id": 1,
      "name": "A",
      "orders": [...]
    }
  ]
}
```

A typical object-mapping pipeline can create:

JSON

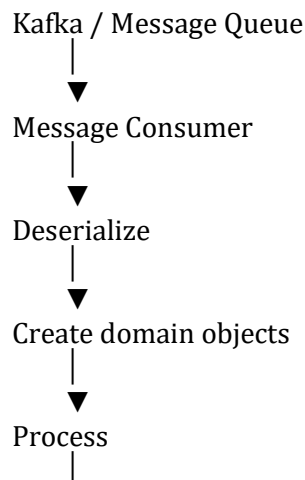
- String objects
- DTO objects
- List objects
- Map objects
- nested domain objects
- temporary parsing objects

If the application processes large volumes of JSON, the number of objects can become very high.

Compact Object Headers can therefore be worth benchmarking.

12. Real-Time Use Case — Messaging System

Consider:





Create response objects

A high-throughput consumer may create millions of objects per second.

For such workloads:

Object count ↑



Header overhead ↑



Compact headers become more valuable

This is especially relevant when heap memory is a limiting factor.

13. Real-Time Use Case — Kubernetes

Suppose a node has:

64 GB RAM

and each Java pod requires:

4 GB

A rough deployment capacity is:

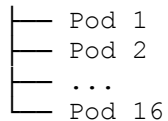
$64 / 4 = 16$ pods

If compact headers reduce actual memory consumption enough to bring the application's practical footprint below the current limit, you might be able to increase deployment density.

Conceptually:

Without compact headers

Node



With lower memory footprint

Node

- Pod 1
- Pod 2
- ...
- Pod 17
- potentially more

This is **not a guaranteed result**; actual container sizing must be based on measurements.

14. Real-Time Use Case — Serverless

Serverless platforms often price or constrain workloads according to memory allocation.

Suppose:

Function

- many temporary objects
- short execution
- memory constrained

Reducing per-object overhead can potentially reduce memory requirements.

This can be valuable because:

Lower memory footprint

- lower memory pressure
- potentially lower GC pressure
- potentially lower infrastructure cost

Again, benchmark the actual function rather than assuming a fixed percentage improvement.

15. Enabling Compact Object Headers

In Java 25:

```
java -XX:+UseCompactObjectHeaders MyApplication
```

The important distinction from Java 24 is that the feature is now a **product feature**, so you no longer need:

```
-XX:+UnlockExperimentalVMOptions
```

Oracle documents `-XX:+UseCompactObjectHeaders` as the Java 25 option. ([Oracle Docs](#))

Recommended testing

Run:

```
java -XX:+UseCompactObjectHeaders \  
-Xms4g \  
-Xmx4g \  
-jar application.jar
```

and compare against:

```
java \  
-Xms4g \  
-Xmx4g \  
-jar application.jar
```

Measure:

Metric	Why
RSS	Actual process memory
Heap used	JVM heap consumption
Allocation rate	Object creation
GC frequency	Memory pressure
GC pause time	Application impact
Throughput	Overall performance
p95/p99 latency	User-visible performance
CPU	Possible secondary effects

16. Important: Product Feature ≠ Default Feature

This is an important Java 25 distinction.

Java version	Status
Java 24	Experimental

Java 25	Product feature
Default in Java 25	No
Enable explicitly	-XX:+UseCompactObjectHeaders

JEP 519 explicitly states that making compact object headers the default object-header layout was **not a goal**. ([OpenJDK Bug System](#))

So don't assume:

Java 25
↓
Compact headers automatically enabled

Instead:

Java 25
↓
Product feature available
↓
-XX:+UseCompactObjectHeaders
↓
Compact headers enabled

17. Compact Headers and Garbage Collection

Compact Object Headers are **not a garbage collector**.

They can, however, influence GC indirectly.

Smaller objects
↓
Smaller live set
↓
Potentially less memory pressure
↓
Potentially different GC behavior

Therefore:

Statement	Correct?
------------------	-----------------

Compact headers replace G1	✗
Compact headers replace ZGC	✗
Compact headers reduce object size	✓
Compact headers can reduce heap usage	✓
Compact headers can indirectly affect GC	✓
Compact headers guarantee fewer GC pauses	✗

Oracle specifically describes reduced live-data size as a benefit, which can influence memory and GC behavior. ([Oracle Docs](#))

18. Compact Object Headers vs Compressed Class Pointers

These are related but different JVM mechanisms.

Aspect	Compact Object Headers	Compressed Class Pointers
Main goal	Reduce total object-header size	Compress class metadata references
Header	64-bit compact layout	Part of traditional header layout
Java 25 feature	JEP 519	Existing JVM optimization
Application code changes	None	None
Memory optimization	★★★★★	★★★★
Object-heavy workload	Excellent	Good
Same feature?	✗	✗
Can interact	Yes	Part of JVM object-layout considerations

Compact Object Headers use more aggressive packing than the traditional compressed-class-pointer object-header layout.

19. Compact Object Headers vs Primitive/Flattened Data Structures

For object-heavy applications, another solution is to reduce the **number of objects themselves**.

Object-heavy design

1 million objects



1 million object headers

More compact representation

Primitive arrays / packed structures



Far fewer object headers

Therefore:

Approach	Best when
Compact Object Headers	You need normal Java objects but have many of them
Primitive arrays	Data is naturally numeric/simple
Flattened/value-oriented structures	Data is suitable for compact representation
Off-heap memory	Very large specialized data structures
Compact Object Headers + better data structures	Often the strongest strategy

20. Alternate Features / Technologies

There is no exact replacement because Compact Object Headers optimize **JVM object representation**.

Requirement	Alternative	When preferable
Reduce object-header	Compact Object Headers	Many small heap objects

overhead		
Reduce number of objects	Primitive arrays / packed structures	Numeric/data-processing workloads
Reduce memory from references	Compressed Oops	Existing HotSpot memory optimization
Reduce memory pressure	Better data structures	Object graph itself is inefficient
Reduce heap allocation	Object reuse/pooling	Carefully controlled high-allocation workloads
Store huge datasets outside heap	Off-heap/direct memory	Specialized high-volume data
Reduce GC impact	G1/ZGC tuning	GC is the actual bottleneck
Reduce object footprint in future designs	Value-object/flattening technologies	When application model supports them
Analyze memory usage	JFR / heap dump	Before deciding on an optimization

21. Compact Object Headers vs Object Pooling

Aspect	Compact Headers	Object Pooling
Reduces header size	✓	✗
Reduces object creation	✗	✓
Reduces live object count	Not necessarily	Potentially
Application code changes	None	Usually required
Complexity	Low	Higher
Risk	Relatively low	Can increase complexity/retention
Best for	Many small objects	Expensive reusable objects
Modern Java recommendation	Often worth testing	Use selectively

In many cases, **don't introduce object pools merely to avoid allocation**. Modern JVMs are very good at short-lived allocations. Compact headers can be attractive because they don't require application-level pooling logic.

22. Compact Object Headers vs GC Tuning

Problem	Compact Headers	GC Tuning
Objects consume too much memory	★★★★★	★★★
Too many small objects	★★★★★	★★★
Long GC pauses	★★★	★★★★★
Allocation rate too high	★★★	★★★★★
Large live set	★★★★★	★★★★★
Wrong heap sizing	★★	★★★★★
Need lower object overhead	★★★★★	★★

The right approach is often:

Measure
↓
Identify memory problem
↓
Determine whether object count is significant
↓
Test compact headers
↓
Tune GC/heap if still necessary

23. A Strong Production Evaluation Strategy

Before enabling it across all production systems:

Step 1 — Establish baseline

Java 25
standard headers

Measure:

Heap
RSS
GC
CPU
Latency
Throughput

Step 2 — Enable

-XX:+UseCompactObjectHeaders

Step 3 — Run identical workload

Same:

- JDK
- CPU
- memory
- traffic
- GC
- application version

Step 4 — Compare

Memory ↓ ?
GC ↓ ?
CPU ↓ ?
Latency ↓ ?
Throughput ↑ ?

Step 5 — Roll out selectively

Canary
↓
10%
↓
25%
↓
50%
↓
100%

This is much safer than assuming a benchmark from another application will apply to yours.

24. Overall Assessment

Category	Rating	Assessment
----------	--------	------------

Memory efficiency	★★★★★	Excellent for object-heavy workloads
Small objects	★★★★★	Strongest use case
Large object graphs	★★★★★	Very attractive
Container density	★★★★★	Potentially valuable
Cloud workloads	★★★★★	Strong candidate
Serverless	★★★★	Worth benchmarking
GC pressure	★★★★	Potential indirect benefit
CPU performance	★★★	Workload-dependent
Large primitive arrays	★	Little direct benefit
Off-heap applications	★	Little direct benefit
Application code changes	★★★★★	None
Deployment complexity	★★★★	Simple JVM flag
Portability	★★★	Platform/JVM dependent
Production readiness	★★★★★	Product feature in Java 25

25. Final Decision Table

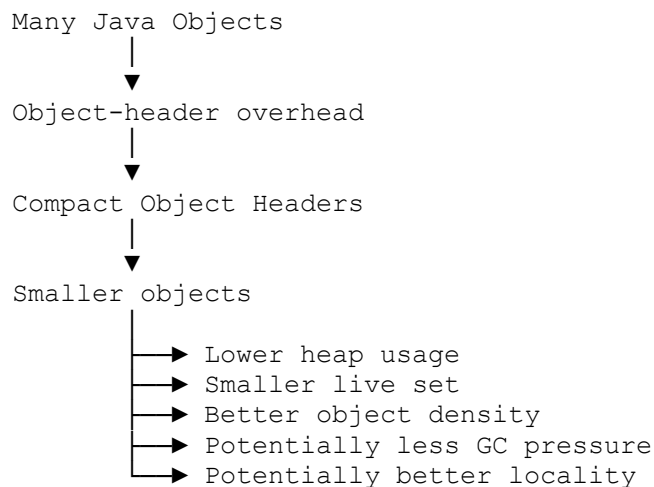
Real-Time Scenario	Recommendation
Millions of small Java objects	🔍 Strongly consider
Large object-heavy microservice	🔍 Strongly consider
Large in-memory cache	🔍 Strongly consider
Graph processing	🔍 Strongly consider
JSON-heavy service	🔍 Consider
High-throughput messaging	🔍 Consider

Kubernetes memory-constrained deployment	🔍 Consider
Serverless Java	🔍 Benchmark and consider
Large AST/compiler structures	🔍 Strong candidate
Application with few objects	🔍 Little benefit
Large primitive arrays dominate memory	🔍 Little direct benefit
Mostly off-heap memory	🔍 Little direct benefit
More than ~4 million loaded classes	🔍 Do not use
Need guaranteed CPU improvement	🔍 Not appropriate
GC pauses are the main problem	🔍 Use GC/JFR analysis too

26. Bottom Line

Compact Object Headers are one of the more interesting Java 25 JVM-level memory optimizations because they reduce the per-object overhead without requiring application-code changes.

The key relationship is:



The strongest candidates are applications with **millions of small, live Java objects**—for example caches, object-heavy microservices, graph processing, messaging, JSON processing, and large domain models. Oracle explicitly identifies reduced object size, live-data size, deployment density, and improved locality as the intended benefits. ([Oracle Docs](#))

Best rule:

If your application's memory usage is driven by the number of Java objects rather than large arrays/off-heap data, benchmark `-XX:+UseCompactObjectHeaders`. If your workload has relatively few large objects, the benefit is likely to be much smaller.

And remember: **Java 25 makes Compact Object Headers a product feature, but it does not make them the default layout—you explicitly enable them with `-XX:+UseCompactObjectHeaders`.** ([OpenJDK Bug System](#))

Java 25 — JFR Method Timing & Tracing

Aspect	Details
Feature	JFR Method Timing & Tracing
UniqueClassName	JFRMethodTimingTracing
JEP	JEP 520
Java Version	25
Year	2025
Status	Final / Product Feature
Technology	Java Flight Recorder (JFR)
Mechanism	Bytecode instrumentation
Main purpose	Measure and trace selected Java method invocations
Primary events	jdk.MethodTiming, jdk.MethodTrace
Best for	Finding slow methods, call relationships, bottlenecks and execution paths
Application code changes	Normally none for the target methods

Performance nature	Diagnostic/profiling feature, not an application optimization
--------------------	---

Java 25 extends JFR with method timing and tracing through **bytecode instrumentation**. Oracle specifically describes the feature as useful for identifying performance bottlenecks, optimizing code, and finding the root causes of bugs. ([Oracle Docs](#))

1. What Is JFR Method Timing & Tracing?

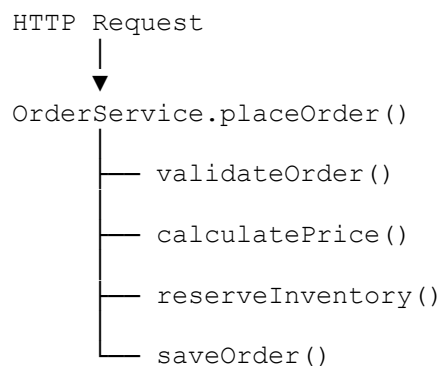
Traditional JFR profiling gives you information such as:

CPU samples
GC
allocation
locks
I/O
threads
exceptions

JFR Method Timing & Tracing adds a more direct question:

"Which specific methods were called, how long did they take, and how did they relate to other method calls?"

Conceptually:



With method timing/tracing, you can investigate something like:

```

placeOrder()      120 ms
├── validateOrder()  2 ms
├── calculatePrice()  5 ms
├── reserveInventory() 98 ms ← bottleneck
└── saveOrder()     15 ms
  
```

This is different from statistical sampling: **the selected method invocations are instrumented so their execution can be recorded.**

2. Method Timing vs Method Tracing

Feature	JFR Method Timing	JFR Method Tracing
Primary question	How long did methods take?	Which methods were called and in what sequence?
Records invocation	Yes	Yes
Duration	Yes	Can provide invocation timing/context
Call relationships	Useful	Primary purpose
Performance analysis	★★★★★	★★★★
Call-flow analysis	★★★★	★★★★★
Debugging execution path	★★★	★★★★★
Potential event volume	High if broadly enabled	Potentially very high
Best usage	Targeted performance investigation	Targeted execution-flow investigation

The JFR API exposes configurable filtering for `jdk.MethodTrace` and `jdk.MethodTiming`, including class, method, and annotation-based filters. ([Oracle Docs](#))

3. Why Is This Useful?

Suppose you have:

```
public void processOrder(Order order) {  
    validate(order);  
    calculatePrice(order);  
    reserveInventory(order);  
    save(order);  
}
```

A normal CPU sample might tell you:

```
reserveInventory()  
  72% of sampled CPU
```

But method timing can answer:

```
reserveInventory()  
  calls: 25,000  
  total duration: 12.5 seconds  
  average: 0.5 ms  
  max: 150 ms
```

This makes the feature particularly useful when the question is:

"How much time is this particular method actually taking?"

rather than:

"Where is the JVM spending sampled CPU time?"

4. Advantages

Advantage	Explanation	Practical value
Method-level visibility	Directly observes selected method invocations	Excellent for targeted investigation
Measures method duration	Identifies slow method executions	Finds latency bottlenecks
Supports method tracing	Shows execution flow among instrumented methods	Useful for debugging complex call paths
Uses JFR	Results are part of the JFR ecosystem	Works with existing JFR/JMC workflows
No source-code logging required	Instrumentation can be applied without manually adding timers everywhere	Less invasive than adding logging
Targeted filtering	Methods/classes/annotations can be selected	Avoids instrumenting the entire application
Useful for production	Can be enabled for focused	Helps diagnose real incidents

diagnosis	investigations	
Good for legacy applications	You don't need to modify every method with timing code	Valuable for older systems
Helps identify root causes	Timing + execution flow provides context	Better than simple log timestamps
Complements sampling	Gives detail where sampling may not be sufficient	Strong combined profiling strategy
Useful for optimization	Shows which methods deserve optimization	More focused engineering effort
Useful for debugging	Method execution order can expose unexpected behavior	Helps reproduce difficult problems

Oracle specifically states that method timing and tracing can identify performance bottlenecks, optimize code, and help find root causes of bugs. ([Oracle Docs](#))

5. Disadvantages

Disadvantage	Explanation	Impact
Instrumentation overhead	Bytecode instrumentation adds work around selected method invocations	Can affect application performance
Can generate many events	Frequently called methods may produce huge amounts of data	Recording size increases
Not suitable for tracing everything	Instrumenting huge portions of an application can become expensive	Target carefully
Can distort very short methods	Instrumentation overhead is proportionally more significant for tiny methods	Measurements need interpretation
Not a replacement for sampling	It doesn't provide the same low-overhead statistical overview	Use sampling for broad profiling

Configuration matters	Poor filtering can produce excessive data	Requires careful JFR settings
Tracing can become noisy	Deep call trees can be difficult to analyze	Focus on relevant packages/classes
Not distributed tracing	It operates inside the JVM	Doesn't replace OpenTelemetry
Not a profiler for every problem	CPU, GC, allocation, locks and I/O may require other JFR events	Combine with other diagnostics
Production overhead must be measured	"JFR is low overhead" does not mean arbitrary method instrumentation is free	Benchmark configuration
High-frequency methods are problematic	get(), getters, utility methods etc. can be called millions of times	Event explosion
Method duration isn't necessarily CPU time	A method can spend time blocked or waiting	Don't confuse wall-clock timing with CPU consumption

The JFR API itself notes that gathering event data can be expensive and provides mechanisms such as `shouldCommit()` and thresholds to control recording overhead. ([Oracle Docs](#))

6. When to Use

Scenario	Recommendation	Why
Find a slow business method	🔍 Excellent	Direct method timing
Investigate API latency	🔍 Excellent	Trace request processing methods
Debug unexpected call sequence	🔍 Excellent	Method tracing
Analyze legacy application	🔍 Excellent	No need to manually add timers everywhere
Investigate a production performance regression	🔍 Excellent	Targeted instrumentation
Identify expensive	🔍 Excellent	Method-level measurements

service methods		
Investigate framework behavior	🔍 Good	Trace selected framework methods
Analyze a specific library	🔍 Good	Filter selected classes/methods
Investigate intermittent slowdowns	🔍 Good	Capture actual invocation timing
Broad CPU profiling	🔍 Use JFR sampling first	Lower overhead
Distributed request tracing	🔍 Use OpenTelemetry	JFR is JVM-local
GC investigation	🔍 Use GC/JFR events	Method tracing isn't the primary tool
Memory leak investigation	🔍 Use heap analysis/JFR	Method tracing isn't sufficient

7. When Not to Use

Scenario	Don't use it as primary tool	Better alternative
Need overall CPU hotspots	🔍	JFR CPU sampling
Need exact CPU consumption	🔍	JFR CPU-time profiling where appropriate
Need distributed tracing	🔍	OpenTelemetry
Need memory-leak analysis	🔍	Heap dump/JFR allocation analysis
Need GC analysis	🔍	JFR GC events + GC logs
Need lock analysis	🔍	JFR monitor/lock events
Need every method traced	🔍	Usually too much data
Extremely high-	🔍	Sampling or selective

frequency methods		instrumentation
Need production-wide permanent tracing	?	Distributed tracing/metrics
Need native-code stack profiling	?	async-profiler/native profiling
Simple application with no performance issue	?	Don't instrument unnecessarily

8. Real-Time Use Case — REST API Bottleneck

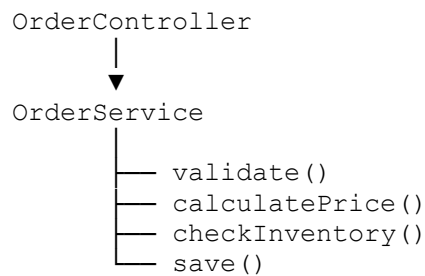
Suppose:

POST /orders

takes:

800 ms

Your service looks like:



You suspect `checkInventory()`.

Instead of adding:

```
long start = System.nanoTime();
```

```
checkInventory();
```

```
long end = System.nanoTime();
```

```
log.info("checkInventory={} ns", end - start);
```

you can use targeted JFR method timing.

Conceptually the result might look like:

```
OrderService.process()
  800 ms
├── validate()      4 ms
├── calculatePrice() 8 ms
├── checkInventory() 730 ms ← bottleneck
└── save()          40 ms
```

Now the investigation becomes much more focused.

9. Real-Time Use Case — Unexpected Method Call

Imagine:

```
public void checkout(Order order) {
    paymentService.charge(order);
    inventoryService.reserve(order);
}
```

But production tracing reveals:

```
checkout()
├── charge()
├── fraudCheck()
├── externalRiskEngine()
├── audit()
└── reserve()
```

The unexpected calls may explain why checkout latency increased.

This is where **method tracing** can be more useful than simply looking at CPU samples.

10. Real-Time Use Case — Legacy Application

Suppose you inherit:

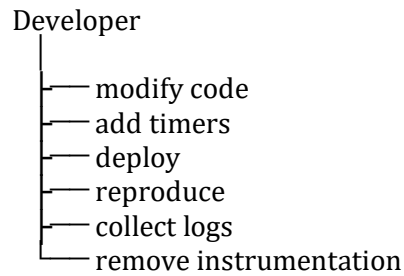
15-year-old Java application

with:

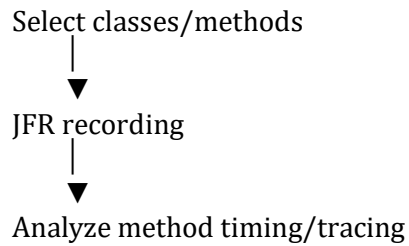
5,000 classes
200,000 methods

Adding manual instrumentation everywhere is impractical.

Without JFR method instrumentation:



With targeted JFR instrumentation:



This can dramatically shorten the diagnostic cycle.

11. Real-Time Use Case — Database Service Layer

Suppose:

```
OrderService
├─ repository.save()
│   └─ transaction()
│       └─ JDBC operations
```

You observe:

repository.save()
220 ms

But:

```
transaction()
  210 ms
```

and:

```
validation()
  2 ms
```

Now you know that optimizing Java-side validation won't solve the problem.

You can then investigate:

- database latency,
- connection pool,
- transaction locks,
- network latency,
- SQL execution.

This illustrates an important principle:

Method timing identifies where to investigate; it doesn't automatically explain why the method is slow.

12. Real-Time Use Case — Recursive Algorithm

Consider:

```
long fibonacci(int n) {
  if (n <= 1) return n;
  return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Tracing can reveal:

```
fibonacci(5)
├─ fibonacci(4)
│  └─ fibonacci(3)
│     └─ fibonacci(2)
├─ fibonacci(3)
│  └─ fibonacci(2)
│     └─ fibonacci(1)
```

This makes the excessive recursive call pattern obvious.

For algorithmic debugging, method tracing can therefore be extremely useful.

13. Real-Time Use Case — Virtual Threads

Java 25 applications may use large numbers of virtual threads.

For example:

```
10,000 virtual threads
├── processRequest()
├── loadUser()
├── calculate()
└── save()
```

JFR already provides dedicated virtual-thread events, including `jdk.VirtualThreadStart`, `jdk.VirtualThreadEnd`, and `jdk.VirtualThreadPinned`. ([Oracle Docs](#))

Method timing/tracing can complement these events when you need to answer:

Which application methods
are taking the time?

rather than simply:

Which virtual threads exist?

14. Real-Time Use Case — Find a Regression

Suppose version 1:

`processOrder()` = 40 ms

Version 2:

`processOrder()` = 120 ms

Use method timing:

Version 1

<code>validate()</code>	5 ms
<code>calculate()</code>	10 ms
<code>persist()</code>	20 ms

Version 2

```
validate()      6 ms
calculate()     70 ms ← regression
persist()      25 ms
```

You immediately know where to investigate.

This is much more actionable than:

CPU usage increased by 15%

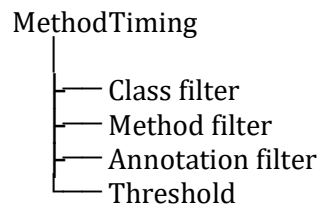
15. JFR Configuration Concept

JFR event settings support:

- enabled
- threshold
- period
- stackTrace
- throttle
- filter

and the filter setting can target classes, methods, and annotations for `jdk.MethodTrace` and `jdk.MethodTiming`. ([Oracle Docs](#))

Conceptually:



This is important because you generally **don't want to instrument every method in a large application.**

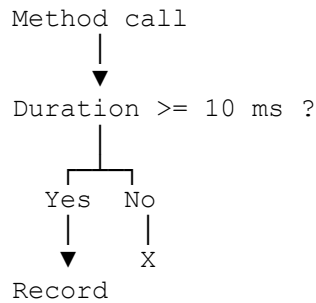
16. Thresholds Are Extremely Important

Suppose you only care about methods taking at least:

10 ms

A threshold can help eliminate insignificant events.

Conceptually:



JFR supports event thresholds and settings through `EventSettings`, including `withThreshold(...)`.
([Oracle Docs](#))

For example, conceptually:

```
recording.enable("jdk.MethodTiming")
    .withThreshold(Duration.ofMillis(10));
```

The exact event/filter configuration should be validated against the JDK 25 JFR configuration being used.

17. JFR Method Timing vs JFR CPU Sampling

This is one of the most important comparisons.

Aspect	Method Timing & Tracing	CPU Sampling
Mechanism	Bytecode instrumentation	Sampling
Method-specific	★★★★★	★★★
Broad CPU hotspot discovery	★★★	★★★★★
Invocation duration	★★★★★	★★
Call-flow analysis	★★★★★	★★★
Overhead	Potentially higher for instrumented methods	Generally lower
Best for	Targeted investigation	Broad profiling
Production continuous	Selectively	Excellent candidate

use		
Finds rarely sampled methods	Better if instrumented	May miss them
Event volume	Can be high	Usually lower
Best strategy	Focused	Broad

18. JFR Method Timing vs JFR Cooperative Sampling

Aspect	Method Timing & Tracing	Cooperative Sampling
JEP	520	518
Main mechanism	Bytecode instrumentation	Cooperative stack sampling
Method invocation timing	★★★★★	★★
Stack hotspot discovery	★★★	★★★★★
Low overhead	★★★	★★★★★
Call tracing	★★★★★	★★
Broad production profiling	★★★	★★★★★
Targeted investigation	★★★★★	★★★★
Best question	"How long does this method take?"	"Where is execution spending time?"

Best combination:

JFR Cooperative Sampling
 +
 JFR Method Timing & Tracing
 ↓
 Broad hotspot discovery
 +
 Targeted method investigation

19. JFR Method Timing vs Logging

Aspect	JFR Method Timing	Manual Logging
Source-code changes	Usually no	Yes
Structured diagnostic data	✓	Depends
Method duration	✓	Manual
Call flow	✓	Manual
Production diagnostics	Strong	Can be noisy
Runtime configuration	Strong	Usually deployment-dependent
Code pollution	Low	High
Debugging convenience	★★★★★	★★★
Business-specific messages	★★	★★★★★
Distributed trace IDs	Not a replacement	Usually requires tracing framework

Manual logs remain better when the question is:

"What business information happened?"

JFR is stronger when the question is:

"What did the JVM/application actually execute?"

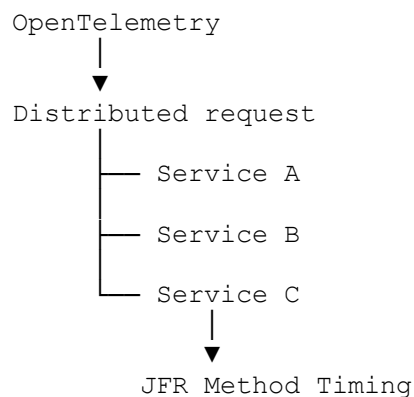
20. JFR Method Timing vs OpenTelemetry

Aspect	JFR Method Timing	OpenTelemetry
JVM method performance	★★★★★	★★★

Distributed tracing	☆☆	☆☆☆☆☆
Cross-service request	✗	✓
Method duration	✓	✓, when instrumented
JVM internals	☆☆☆☆☆	☆☆
GC/threads/JIT/JVM events	☆☆☆☆☆	☆☆
Production observability	☆☆☆☆	☆☆☆☆☆
Root-cause performance analysis	☆☆☆☆☆	☆☆☆☆
Network/service relationships	☆☆	☆☆☆☆☆

They are **complementary**, not competing technologies.

A modern microservice can use:



21. Alternate Features / Technologies

Alternative	Best use case	Advantage over Method Timing & Tracing	Disadvantage
JFR Cooperative Sampling (JEP 518)	Broad CPU/stack profiling	Lower-overhead broad view	Doesn't directly time every selected invocation

JFR CPU-Time Profiling (JEP 509)	CPU-time analysis	More CPU-focused	Not general method tracing
JFR standard events	GC, allocation, locks, I/O, threads	Rich JVM diagnostics	Not arbitrary method tracing
async-profiler	Deep CPU/native profiling	Excellent low-overhead profiling	External tool
OpenTelemetry	Distributed tracing	Cross-service visibility	Not a JVM-internal profiler
Micrometer/metrics	Service-level metrics	Excellent long-term metrics	Less detailed execution flow
Manual System.nanoTime()	Small targeted code section	Very simple	Code changes and maintenance
Aspect-oriented timing	Application method metrics	Flexible business instrumentation	Additional framework/runtime complexity
Java agent instrumentation	Custom tracing/profiling	Highly customizable	More operational complexity
Logging	Business/debug information	Rich contextual messages	High volume/noisy for performance profiling

22. Which Alternative Should You Choose?

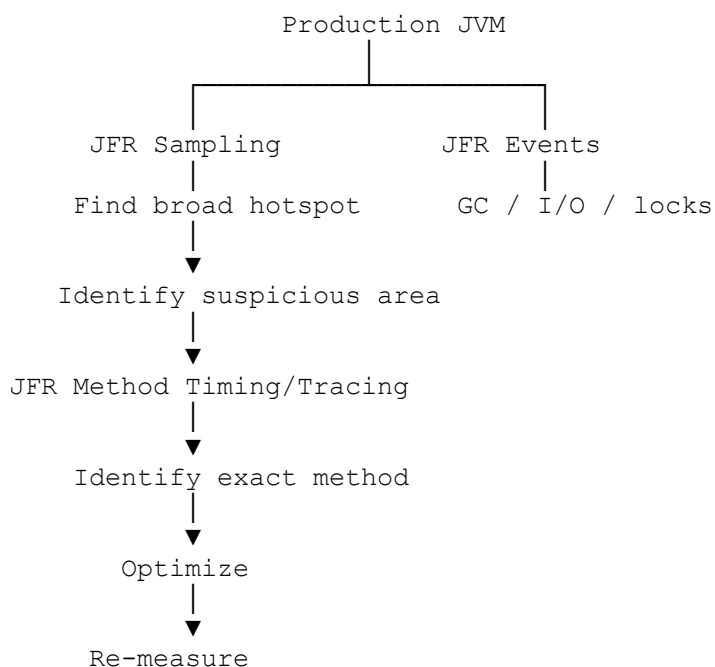
Your actual question	Best tool
Where is CPU time going?	JFR CPU sampling / cooperative sampling
How long does this specific method take?	JFR Method Timing
Which methods call this method?	JFR Method Tracing
Why is a particular API slow?	Method Timing + JFR events
Why is GC slow?	JFR GC events
Why is allocation high?	JFR allocation events

Why is a lock slow?	JFR lock/monitor events
Which service causes end-to-end latency?	OpenTelemetry
Need native CPU profiling?	async-profiler
Need simple application metrics?	Micrometer
Need business-level audit information?	Logging/events

23. Recommended Production Strategy

For a large Java 25 production service, don't start by tracing everything.

Use a layered strategy:



This is generally more efficient than permanently instrumenting thousands of methods.

24. Example Investigation Workflow

Suppose API latency increased from:

p99 = 200 ms

to:

p99 = 700 ms

Stage 1 — Broad JFR

Find:

CPU

GC

locks

I/O

threads

Stage 2 — Identify suspicious service

PaymentService

Stage 3 — Enable targeted method timing

PaymentService.*

Stage 4 — Discover:

```
PaymentService.process()
  680 ms
  └─ validate()      5 ms
  └─ fraudCheck()   620 ms ←
  └─ persist()      40 ms
```

Stage 5 — Investigate fraud check

Maybe:

```
fraudCheck()
└─ externalRiskService()
    └─ network latency
```

The method profiler has not "fixed" the application.

It has **reduced the search space dramatically**.

25. Important Performance Rule

Don't make this mistake:

```
Enable MethodTiming for every method
      ↓
Collect enormous JFR recording
      ↓
Application overhead increases
      ↓
Analysis becomes difficult
```

Prefer:

```
Broad sampling
      ↓
Find suspicious package/class
      ↓
Target specific methods
      ↓
Apply threshold/filter
      ↓
Record short diagnostic window
```

This follows the general JFR principle that event data collection has a cost and that settings such as thresholds, filters and stack traces should be configured deliberately. ([Oracle Docs](#))

26. Overall Assessment

Category	Rating	Assessment
Method-level timing	★★★★★	Excellent
Call-flow investigation	★★★★★	Excellent
Performance debugging	★★★★★	Excellent
Production diagnostics	★★★★	Very good when targeted
Broad CPU profiling	★★★	Use sampling instead
Distributed tracing	★★	Use OpenTelemetry
JVM diagnostics	★★★★★	Excellent as part of JFR
Ease of adoption	★★★★★	Built into JDK/JFR

Application code changes	☆☆☆☆	Usually unnecessary
Overhead when narrowly targeted	☆☆☆	Generally manageable
Overhead when broadly instrumented	☆☆	Can become significant
Long-term continuous use	☆☆	Depends heavily on filtering
Root-cause investigation	☆☆☆☆	Very useful

27. Final Decision Table

Scenario	Recommendation
Find slow business methods	🔍 Use
Investigate API latency	🔍 Use
Investigate method call sequence	🔍 Use
Legacy application performance investigation	🔍 Use
Production performance regression	🔍 Use selectively
Investigate specific framework/library behavior	🔍 Use selectively
Need broad CPU hotspot discovery	🔍 Prefer JFR sampling
Need low-overhead continuous profiling	🔍 Prefer cooperative sampling
Need distributed tracing	🔍 Use OpenTelemetry
Need GC analysis	🔍 Use JFR GC events
Need memory leak investigation	🔍 Use heap/JFR allocation analysis
Need native stack profiling	🔍 Consider async-profiler
Trace every method in a huge application	🔍 Avoid

Instrument extremely hot tiny methods	⚠ Avoid unless necessary
---------------------------------------	--------------------------

Bottom line

JFR Method Timing & Tracing is a targeted diagnostic tool, not a replacement for general-purpose profiling.

Its strongest value is:

```

Broad JFR profiling
↓
Find suspicious component
↓
JFR Method Timing & Tracing
↓
Find exact method
↓
Measure duration / call path
↓
Fix bottleneck
↓
Verify with JFR

```

For Java 25 production systems, a particularly strong combination is:

JFR Cooperative Sampling + JFR Method Timing & Tracing + normal JFR events.

Use **Cooperative Sampling** to discover *where* the application is spending time, then use **Method Timing & Tracing** to investigate *which specific methods and call paths* are responsible. JEP 520 specifically adds method timing/tracing through bytecode instrumentation, while JFR's existing event configuration supports thresholds, stack traces, throttling and method/class filters. ([Oracle Docs](#))

Java 25 — Generational Shenandoah

Aspect	Details
Feature	Generational Shenandoah
UniqueClassName	GenerationalShenandoah

JEP	JEP 521
Java Version	25
Year	2025
Status	Final / Product Feature
Introduced experimentally	JDK 24 / JEP 404
Primary technology	Shenandoah Garbage Collector
Main option	-XX:+UseShenandoahGC -XX:ShenandoahGCMode=generational
Main goal	Combine Shenandoah's low-pause concurrent collection with generational GC
Key idea	Collect young objects more frequently and old objects less frequently
Application code changes	None
Main benefit	Better efficiency for workloads with many short-lived objects while retaining Shenandoah's low-pause characteristics
Default in Java 25	No — generational mode became a product feature, but was not made the default by JEP 521
Best candidates	Large heaps, high allocation rates, latency-sensitive services
Main alternatives	G1 GC, ZGC, non-generational Shenandoah

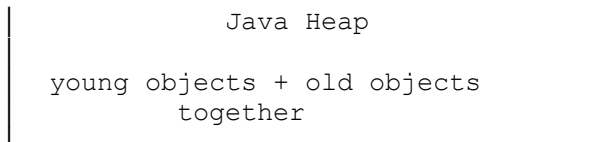
JEP 521 changes Shenandoah's generational mode from experimental to a product feature in JDK 25. The mode is selected with -XX:ShenandoahGCMode=generational; JEP 521 specifically did **not** make it the default. ([OpenJDK Bug System](#))

1. What Is Generational Shenandoah?

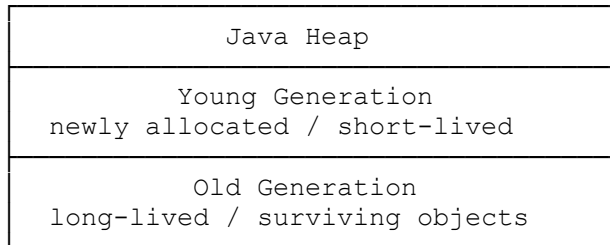
The basic observation behind generational GC is:

Most newly created objects die young, while objects that survive several collections are more likely to live for a long time.

A traditional non-generational Shenandoah treats the heap as one generation:



Generational Shenandoah separates the heap conceptually into:



The collector can then concentrate more effort on the young generation, where there is generally more garbage.

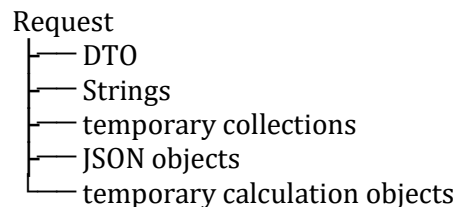
OpenJDK describes Generational Shenandoah as retaining Shenandoah's concurrent collection while dividing the heap into young and old generations. Its young-generation collections and evacuation remain concurrent, unlike the traditional stop-the-world young-generation evacuation model used by G1. ([OpenJDK Mail](#))

2. Why Generational GC Helps

Consider a web server processing:

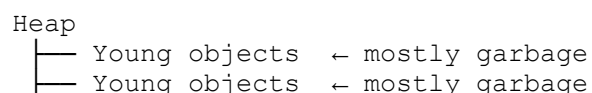
10,000 requests/second

Each request creates:



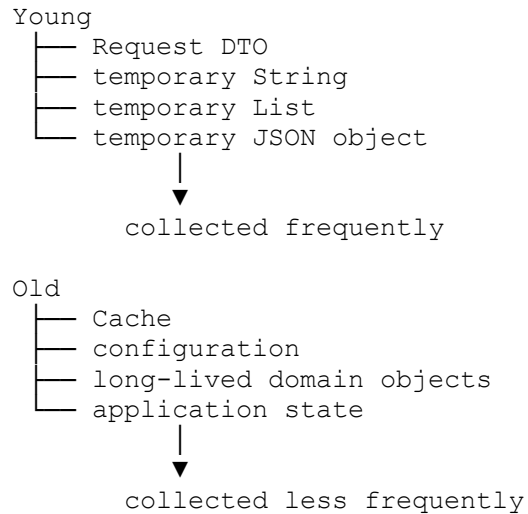
Most of these objects may live only for milliseconds.

Without generational awareness:



- Old objects
- Old objects
- Young objects ← mostly garbage

With generations:



The collector can therefore avoid repeatedly processing long-lived objects when the majority of newly allocated objects are already dead.

3. How Generational Shenandoah Differs from G1

Generational Shenandoah is often compared with G1 because both use generations.

However, they are not the same.

Aspect	Generational Shenandoah	G1
Generational	✓	✓
Concurrent GC	✓	✓
Low-pause goal	★★★★★	★★★★
Young collection	Concurrent evacuation	Stop-the-world evacuation
Old collection	Concurrent	Concurrent marking + mixed STW evacuation
Very large heap	Excellent candidate	Excellent candidate

Latency sensitivity	★★★★★	★★★★
Throughput	★★★★	★★★★★
Default Java 25 GC	✗	✓
Configuration maturity	High	Very high
Best reason to choose	Very low pause sensitivity	General-purpose balanced GC

The key technical difference is that Generational Shenandoah retains Shenandoah's concurrent evacuation approach; OpenJDK's implementation description explicitly contrasts this with G1. ([OpenJDK Mail](#))

4. Advantages

Advantage	Explanation	Real-world benefit
Low-pause collection	Shenandoah performs most GC work concurrently	Good for latency-sensitive services
Generational efficiency	Young objects are collected more frequently	Less unnecessary processing of old objects
Excellent for high allocation rates	Many short-lived objects can be handled efficiently	Useful for APIs, messaging, JSON processing
Smaller effective GC workload	Old objects don't need to be considered as frequently as young objects	Potentially lower GC overhead
Better memory efficiency	Dead young objects can be reclaimed quickly	Helps memory-constrained workloads
Large-heap friendly	Shenandoah is designed for large heaps	Suitable for memory-intensive services
Concurrent evacuation	Object movement can happen concurrently	Helps maintain low pauses
No application-code changes	JVM-level feature	Easy to evaluate
Product feature in Java 25	No longer experimental	Production-ready JVM option
Adaptive generation sizing	JVM can manage young/old	Less manual sizing

	capacity dynamically	
Good latency/throughput compromise	Generational collection reduces unnecessary old-object work	Potentially better overall efficiency
Useful for modern server workloads	Many modern applications allocate huge numbers of temporary objects	Strong candidate for microservices

OpenJDK's Generational Shenandoah implementation automatically adjusts young- and old-generation sizes to make effective use of heap capacity. ([OpenJDK Mail](#))

5. Disadvantages

Disadvantage	Explanation	Impact
Not automatically the best GC	Performance depends on workload	Benchmark required
Higher implementation complexity	Generational barriers and remembered-set/card-table mechanisms add complexity	More GC internals
Potential barrier overhead	Generational GC needs to track cross-generation references	Can consume CPU
May not benefit non-generational workloads	Applications with unusual object lifetimes may gain little	Could be neutral or worse
Not a zero-pause collector	Shenandoah minimizes pauses; it does not eliminate them	Latency can still vary
Potential throughput trade-off	Concurrent GC and barriers consume CPU	Peak throughput may differ from G1/Parallel
More GC threads can consume CPU	Concurrent work competes with application threads	Important in CPU-constrained containers
Different tuning behavior	Existing Shenandoah tuning assumptions may not transfer directly	Re-test configurations

Monitoring changes	Generational GC introduces additional generation-related metrics/logging	Tools may need adjustment
Not useful for every application	Low-allocation or very small applications may gain little	Complexity without benefit
Heap size still matters	A poor heap configuration can overwhelm any GC strategy	Proper sizing remains essential
Not a memory leak solution	Long-lived accidental references still remain live	Requires heap analysis

OpenJDK notes that generational mode can behave differently from non-generational Shenandoah and that workloads that are inherently non-generational may potentially see some degradation. ([OpenJDK](#))

6. When to Use

Workload	Recommendation	Reason
Large Java heap	👍 Excellent candidate	Shenandoah is designed for large heaps
High allocation rate	👍 Excellent	Young objects can be collected efficiently
Many short-lived objects	👍 Excellent	Strong generational benefit
REST APIs	👍 Consider	Request objects are often short-lived
Microservices	👍 Consider	High allocation + latency requirements
Financial trading services	👍 Consider	Low latency can be important
Messaging systems	👍 Consider	Large numbers of temporary messages
JSON-heavy applications	👍 Consider	Parsing produces many short-lived objects

Large caches	❓ Consider	Large heap + latency sensitivity
Interactive applications	❓ Consider	Long pauses are undesirable
Search/indexing systems	❓ Consider	Large heaps and object churn
Data processing services	❓ Benchmark	Depends on allocation pattern
Memory-constrained containers	❓ Benchmark	GC CPU overhead must be measured
Small command-line application	❓ Usually unnecessary	Shenandoah itself may be excessive

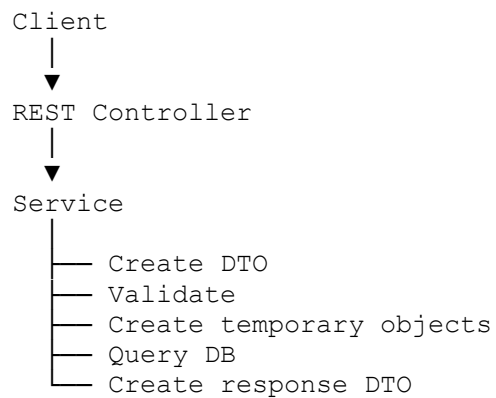
7. When Not to Use

Scenario	Recommendation	Why
Small application	❓ Don't	Simpler GC is usually sufficient
Very small heap	❓ Usually don't	Shenandoah's strengths may not matter
Maximum raw throughput is the only goal	❓ Benchmark G1/Parallel	Concurrent GC has overhead
CPU is severely constrained	❓ Benchmark carefully	Concurrent GC consumes CPU
Allocation rate is very low	❓ Probably unnecessary	Generational benefits may be small
Workload has unusual object lifetimes	❓ Benchmark	Generational assumptions may not hold
Existing G1 already meets latency goals	❓ Don't switch automatically	Migration may provide little benefit
Need simplest JVM configuration	❓ Prefer G1	G1 is the Java default and highly mature
Application has memory leaks	❓ Not the solution	Fix retention/root references

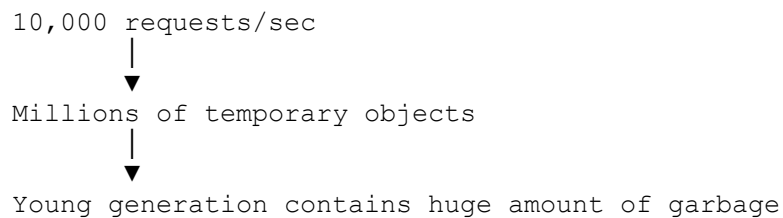
Need deterministic real-time GC	❓ Not sufficient	Shenandoah is low-latency, not hard real-time
--	-------------------------	---

8. Real-Time Use Case — REST Microservice

Consider:



At high traffic:



Generational Shenandoah can focus collection effort on the young generation while concurrently handling the old generation.

A good candidate when:

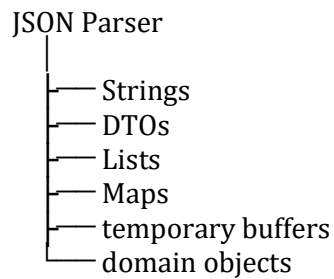
High allocation
+
Large heap
+
Low latency requirement

9. Real-Time Use Case — JSON Processing

Suppose an API receives:

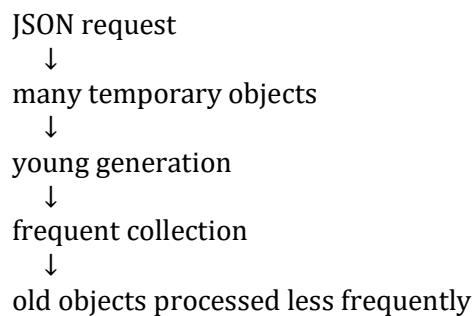
```
{  
  "customer": {  
    "id": 1001,  
    "orders": [...]  
  }  
}
```

Deserialization may create:



Many temporary objects die quickly.

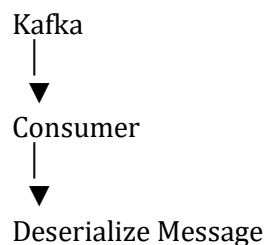
Therefore:

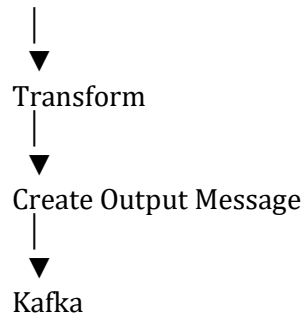


This is a strong generational-GC workload.

10. Real-Time Use Case — Messaging Platform

Consider:





A consumer processing:

500,000 messages/sec

may create huge numbers of short-lived objects.

Generational Shenandoah can be attractive when the application needs:

High allocation throughput

+

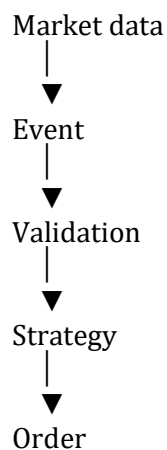
Low pause latency

+

Large heap

11. Real-Time Use Case — Financial Trading Platform

A trading system may have:



Many temporary objects may be generated during event processing.

Long GC pauses can cause:

GC pause
↓
request/event delay
↓
higher latency
↓
potentially missed processing deadlines

Generational Shenandoah is worth benchmarking when low latency is more important than absolute maximum throughput.

However, for true **hard real-time guarantees**, Shenandoah should not be treated as a hard-real-time GC.

12. Real-Time Use Case — Large In-Memory Cache

Imagine:

Heap = 128 GB

with:

Long-lived:

- ├─ cache entries
- ├─ configuration
- └─ indexes

Short-lived:

- ├─ requests
- ├─ temporary lookup objects
- ├─ serialization objects
- └─ intermediate results

This is an excellent situation for generational collection:

128 GB heap

- ├─ Young → frequent collection
- └─ Old → less frequent collection

The important point is that the cache's long-lived objects don't need to be processed as aggressively as the constantly changing request objects.

13. Real-Time Use Case — Search Engine

A search service may maintain:

```
Large indexes
+
Query objects
+
Scoring objects
+
Temporary result sets
```

Conceptually:

```
Long-lived index
└─ Old generation

Query processing
├─ temporary objects
├─ scoring objects
└─ result objects
    └─ Young generation
```

Generational Shenandoah can be attractive when query traffic generates substantial temporary object churn while the index remains long-lived.

14. Real-Time Use Case — Kubernetes

Suppose:

```
Pod
├─ 32 GB heap
├─ high request rate
└─ strict p99 latency target
```

You observe:

Allocation rate: high
GC pauses: problematic
CPU: sufficient
Heap: large

This is a reasonable candidate for:

```
java \
-XX:+UseShenandoahGC \
-XX:ShenandoahGCMode=generational \
```

-jar service.jar

Then compare against the existing collector using identical traffic.

Measure:

Metric	Why
p50 latency	Normal behavior
p95 latency	Tail latency
p99/p999 latency	GC impact
Allocation rate	Object churn
GC CPU	Collector overhead
Pause duration	Latency impact
Throughput	Overall efficiency
RSS	Container memory
CPU throttling	Kubernetes resource interaction

15. How to Enable It in Java 25

In Java 25, generational mode is a product option and no longer requires:

-XX:+UnlockExperimentalVMOptions

The basic configuration is:

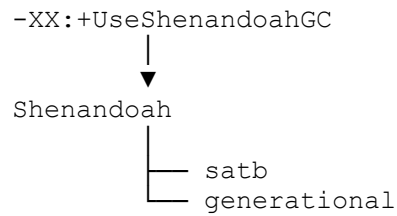
```
java \  
-XX:+UseShenandoahGC \  
-XX:ShenandoahGCMode=generational \  
-jar application.jar
```

OpenJDK's JDK 25 compatibility record explicitly states that generational is no longer experimental and that UnlockExperimentalVMOptions is unnecessary. ([OpenJDK Bug System](#))

16. Important: JEP 521 Did Not Make It the Default

This distinction is important.

Java 25



You explicitly select:

`-XX:ShenandoahGCMode=generational`

JEP 521's purpose was to make the generational mode a **product feature**, not to make it the default mode. ([OpenJDK Bug System](#))

17. Generational Shenandoah vs Non-Generational Shenandoah

Aspect	Generational Shenandoah	Non-generational Shenandoah
Young/old generations	✓	✗
Short-lived-object optimization	★★★★★	★★★
Large heap	★★★★★	★★★★★
Low latency	★★★★★	★★★★★
High allocation rate	★★★★★	★★★★
Simplicity	★★★	★★★★
Long-lived object workload	★★★★	★★★★★
Product feature in Java 25	✓	✓
Main use case	General modern object-	Workloads where generations

	heavy workload	provide little advantage
--	----------------	--------------------------

18. Generational Shenandoah vs G1

Feature	Generational Shenandoah	G1
Generational	✓	✓
Concurrent young evacuation	✓	✗
Concurrent old evacuation	✓	✗
Low pause	★★★★★	★★★★
Throughput	★★★★	★★★★★
Large heap	★★★★★	★★★★★
Mature default	★★★	★★★★★
Tuning complexity	★★★	★★★★
Latency-sensitive service	★★★★★	★★★★
General-purpose service	★★★★	★★★★★
Default Java 25 collector	✗	✓

Oracle documents G1 as the default collector and describes it as a server-style collector targeting large heaps with limited GC latency. ([Oracle Docs](#))

19. Generational Shenandoah vs ZGC

Aspect	Generational Shenandoah	ZGC
Generational	✓	✓ in modern JDKs

Concurrent collection	✓	✓
Low pause	★★★★★	★★★★★
Large heap	★★★★★	★★★★★
Allocation-heavy workload	★★★★★	★★★★★
Maximum latency sensitivity	Excellent	Excellent
Throughput	Very good	Very good
Default	✗	✗
Main distinction	Shenandoah evacuation model	ZGC colored-pointer/barrier architecture

If the primary requirement is extremely low latency, **both deserve benchmarking**.

20. Alternate Features / Technologies

There isn't a single direct replacement for Generational Shenandoah. The alternatives are other garbage collectors.

Alternative	Best for	Choose it when
G1 GC	General-purpose large applications	You want the safest general-purpose choice
ZGC	Very low latency + large heaps	Tail latency is extremely important
Non-generational Shenandoah	Shenandoah workloads without generational benefit	Workload doesn't benefit from generations
Parallel GC	Maximum throughput	Latency requirements are relaxed
Serial GC	Small/simple applications	Heap and workload are small
Object/data-structure optimization	Reducing allocation itself	GC is suffering because application creates too many objects

JFR	GC diagnosis	You need evidence before changing collectors
-----	--------------	--

21. Which Alternative Should You Choose?

Requirement	Best candidate
General enterprise application	G1
Very low latency + large heap	ZGC / Shenandoah
High allocation + low latency	Generational Shenandoah
Large cache + low pauses	Generational Shenandoah / ZGC
Maximum throughput	Parallel GC / G1 benchmark
Small application	G1 / Serial
Huge heap	G1 / ZGC / Shenandoah
Hard real-time requirement	Neither Shenandoah nor ZGC guarantees hard real-time behavior
GC overhead caused by excessive allocations	Fix application allocation patterns first

22. A Practical GC Selection Matrix

Application characteristic	G1	Generational Shenandoah	ZGC	Parallel
Small heap	?	?	?	?
Large heap	?	?	?	?
Huge heap	?	?	?	?
High allocation	?	??	??	?

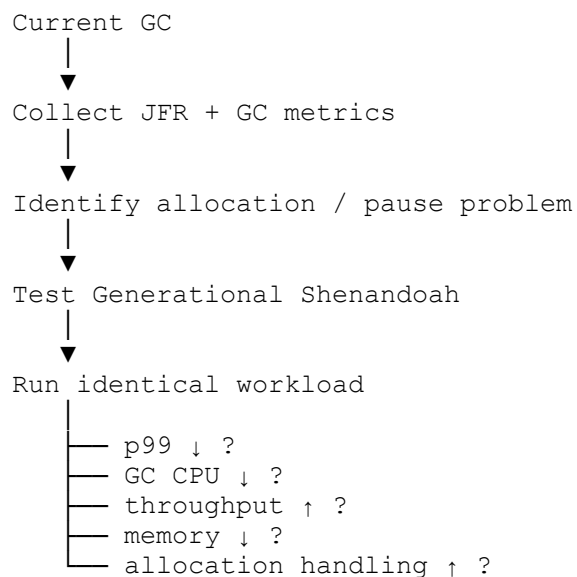
Short-lived objects	??	??	??	?
Very low latency	?	??	??	?
Maximum throughput	??	?	?	??
General-purpose	??	?	?	?
Minimal tuning	??	?	?	?
Container microservices	?	??	??	?
Financial/trading latency	?	??	??	?

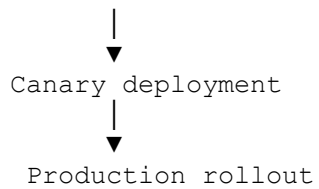
?? means particularly strong candidate; it does **not** mean guaranteed superiority.

23. Recommended Production Evaluation

Do not switch a production JVM to Generational Shenandoah simply because it is newer.

Use this process:





24. What You Should Measure

Metric	Desired result
p99 latency	↓
p999 latency	↓
GC pause time	↓
GC CPU consumption	↓ or acceptable
Application CPU	↓ or acceptable
Throughput	↑ or unchanged
Allocation throughput	Stable
Heap occupancy	↓ or stable
RSS	↓ or stable
OOM events	0
Tail latency during allocation spikes	Improved
Application error rate	0 increase

A GC is successful only if the **application-level metrics** improve or remain acceptable.

25. Overall Assessment

Category	Rating	Assessment
Low-pause GC	★★★★★	Excellent

High allocation workloads	★★★★★	Excellent
Short-lived objects	★★★★★	Excellent
Large heaps	★★★★★	Excellent
Latency-sensitive services	★★★★★	Excellent candidate
Cloud microservices	★★★★★	Strong candidate
Messaging	★★★★★	Strong candidate
JSON-heavy applications	★★★★★	Strong candidate
Maximum raw throughput	★★★★	Benchmark against G1/Parallel
Small applications	★★	Usually unnecessary
CPU-constrained environment	★★★	Benchmark carefully
Configuration simplicity	★★★	More specialized than G1
Production readiness	★★★★★	Product feature in Java 25

26. Final Decision Table

Real-Time Scenario	Recommendation
Large heap + millions of short-lived objects	🔍 Strongly consider
High-throughput REST API	🔍 Consider
Latency-sensitive microservice	🔍 Strongly consider
JSON-heavy service	🔍 Strong candidate
High-volume messaging	🔍 Strong candidate
Large in-memory cache	🔍 Strong candidate

Search/indexing service	⌚ Consider
Financial/trading workloads	⌚ Benchmark seriously
Kubernetes large-heap service	⌚ Consider
Small CLI application	⌚ Usually don't use
Very small heap	⌚ Usually don't use
CPU-starved container	⌚ Benchmark carefully
Maximum throughput only	⌚ Compare with G1/Parallel
Existing G1 already meets requirements	⌚ Don't switch without evidence
Hard real-time requirements	⌚ Not a hard-real-time solution
Memory leak	⌚ Fix object retention instead

27. Bottom Line

Generational Shenandoah is essentially a combination of two important ideas:

Generational GC
 +
 Shenandoah concurrent collection
 ↓
 Generational Shenandoah

Its sweet spot is:

Large Java applications that allocate many short-lived objects while requiring consistently low GC pauses.

For example:

```
REST API
+
high request rate
+
many temporary objects
+
large heap
+
strict p99 latency
```

↓
Generational Shenandoah

JEP 521 made this mode a **non-experimental product feature in Java 25**. It did **not** make it the default Shenandoah mode. ([OpenJDK Bug System](#))

My practical recommendation

Situation	Choice
General enterprise application	Start with G1
Very low latency + large heap	Benchmark ZGC and Generational Shenandoah
High allocation + many short-lived objects + low latency	Generational Shenandoah is a very strong candidate
Existing Shenandoah deployment	Benchmark generational mode
Small/simple application	Stay with G1/Serial as appropriate
Maximum throughput	Benchmark G1/Parallel before choosing Shenandoah

One particularly important point: **don't choose Generational Shenandoah merely because it is generational**. Choose it when measurements show that its combination of **concurrent collection + generational efficiency + low pause behavior** fits your application's allocation and latency profile.